Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh City, October 2026 –

Table of Contents

I. Record of Changes 3

II. Software Design Document 4

1. System Design 4

1.1 System Architecture 4

1.2 Technology Declaration 6

1.3 Package Diagram 9

2. Database Design 14

2.1 Logical ERD 14

2.2 Table Descriptions 21

2.3 Database Script 25

3. Detailed Design 25

4. Business Flows and Entity States 26

4.1 Activity Diagrams 26

4.2 State Machine Diagrams 36

I. Record of Changes

Date	A* M, D	In charge	Change Description
26/10/2026	A	Nguyen Binh Khiem	Created the SDD draft for Review 2: system architecture, technology declaration, package diagrams of the back-end, web and mobile applications, logical ERD by business area with the table descriptions, database script, and state machines of the main entities. The detailed design and the activity diagrams of the five main workflows are assigned to their owners.
30/10/2026	M	Nguyen Binh Khiem	Declared Vercel as the host of the web application on staging: third-party services, deployment environments and the system architecture diagram.
01/11/2026	A	Nguyen Binh Khiem	Added the activity diagrams of the five main workflows (WF-01 to WF-05), drawn by the workflow owners, with an explanation of each flow in section 4.1. The later figures are renumbered.
02/11/2026	M	Nguyen Binh Khiem	Aligned the mobile application with Report 1 (LI-8): the mobile application serves customers and hotel staff, and staff get the same functions as on the web, including the QR check-in.
03/11/2026	M	Nguyen Binh Khiem	Replaced Figure 1 with the updated system architecture diagram (Frontend, Backend, Data Layer and six third-party services) and rewrote section 1.1 to match it. Added the planned maps and social media services to section 1.2.1. Updated section 1.3.3 now that the staff screens of the mobile application exist.
03/11/2026	M	Nguyen Binh Khiem	Aligned the document with the review-2 tag: the database script is rebuilt from the fourteen migrations; the state machines of booking, room and room night and the back-end and mobile package diagrams show the implemented check-in, check-out, no-show, notifications, dashboard and staff screens; the diagram sources are committed in docs/diagrams.
03/11/2026	M	Nguyen Binh Khiem	Updated Figure 1 with the Recommendation Service (offline batch training, planned) and Cloudinary, and added both to the component table of section 1.1 together with the recommendation flow.
04/11/2026	M	Nguyen Binh Khiem	Redrew Figure 23 as a state machine of booking.status with its seven states (the invoice states stay in Figure 24) and added the link column of notification to Figure 5. The PlantUML sources in docs/diagrams now match every figure of this document.

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

StayReco follows a three-layer client and server architecture, shown in Figure 1. The Frontend layer has two client applications, a Next.js web application and an Expo mobile application. Both send HTTPS requests to the StayReco Service, a NestJS REST API under /api/v1 that owns all business rules. The Backend layer also holds the Recommendation Service, which is planned for phase 5 and works offline: it reads interaction data from the primary PostgreSQL database in the Data Layer, trains the model and saves the recommendation results back. The StayReco Service reads those saved results when it ranks hotels for a customer, so the two services share data only through the database and never call each other. Third-party services are called only by the StayReco Service over HTTPS; the one exception is the VietQR image, which the payment page loads directly from SePay. Components that appear in the figure but are not built yet are marked as planned in the table below.

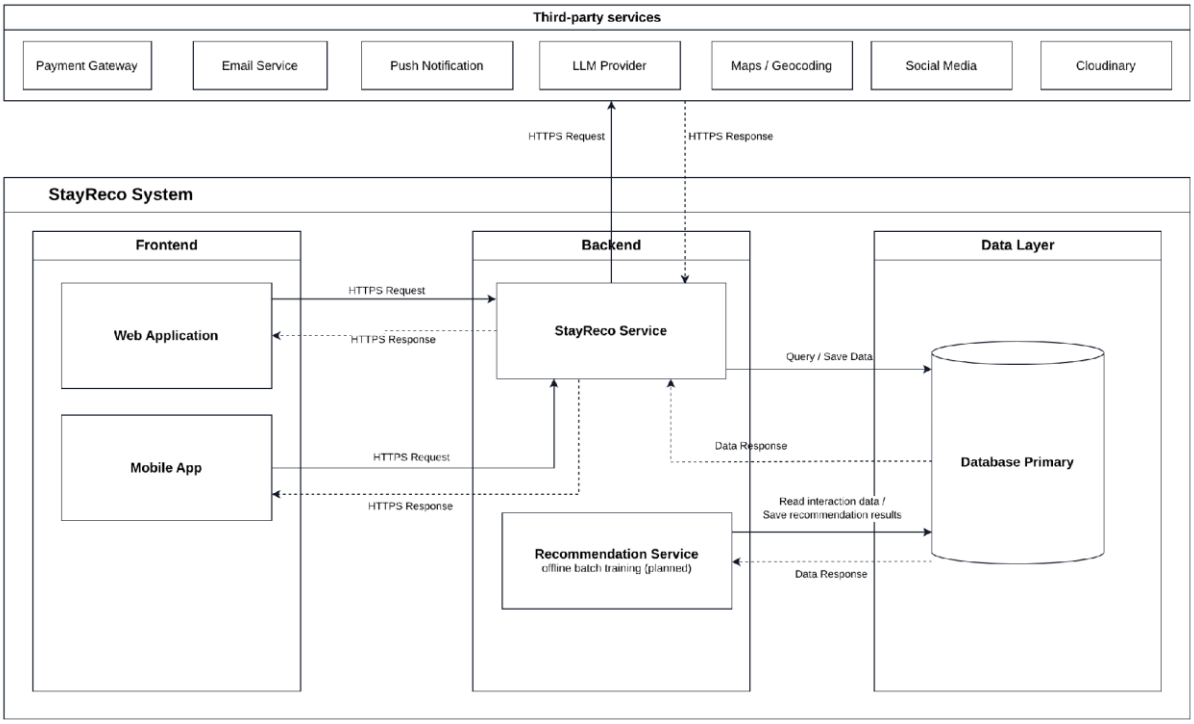


Figure 1. StayReco system architecture

The role of each component in the diagram is as follows.

#	Component	Description
1	Web Application	Next.js 16 application used by every role: public pages for guests, the customer account, and the workspaces of partners, staff, platform managers and admins. It sends HTTPS requests to the StayReco Service with Axios and TanStack Query.
2	Mobile App	Expo application for customers (search, booking, payment, trips, assistant) and for hotel staff, who get the same functions as on the web, including scanning the QR voucher at check-in. The staff screens were added in November;

		some of them still show sample data and will be connected to the API in the next sprint. Tokens are kept in the secure store of the device.
3	StayReco Service	NestJS 11 service and the only back-end of the system. It authenticates with JWT access tokens and rotating refresh tokens, checks roles and hotel access, validates every request with the shared zod schemas, applies the business rules and answers with the common response envelope.
4	Recommendation Service (planned)	Offline batch job, planned for phase 5, that trains the recommendation model. It reads interaction data such as bookings, searches and reviews from the database and saves the recommendation results back. It serves no request from the applications and never calls the StayReco Service; the StayReco Service only reads the saved results.
5	Background jobs (inside the StayReco Service)	Timers that run in the same process as the StayReco Service, which is why the figure does not draw them as a separate box. One releases unpaid holds after 15 minutes, one approves undecided refund requests when the check-in time is reached, one marks paid bookings never checked in as no-show at the check-out time, and one moves the hotel's net amount from pending to available after the holding period.
6	@stayreco/shared (used by Frontend and Backend)	Package in the monorepo with the zod schemas, enums, error codes and response types. The web, the mobile application and the StayReco Service build against it, so requests and responses are checked by the same rules on both sides. It is a build-time package, not a running component, so it is not drawn in the figure.
7	Database Primary	PostgreSQL 16 with pgvector (embeddings), pgcrypto (UUID keys) and unaccent (search without Vietnamese accents). The StayReco Service queries and saves data through Prisma, and the schema is managed by Prisma migrations. Staging runs one primary instance on Render and has no read replica.
8	Payment Gateway (SePay)	The customer transfers money to the platform bank account (ACB) with the booking code as content. SePay reads the incoming transaction and calls the webhook of the StayReco Service, authenticated with an API key.
9	Email Service (Gmail SMTP)	Sends the email verification codes, password reset and invitation links, and the booking and refund emails through Nodemailer.

10	Cloudinary	Stores hotel and room images and the verification documents of partners. The StayReco Service uploads the files and serves the documents through signed URLs.
11	Push Notification (planned)	Firebase Cloud Messaging is planned for pushing booking and check-in reminders to the mobile application. For now notifications are stored in the database and shown inside the web and mobile applications.
12	LLM Provider (planned)	The OpenAI API is planned for the AI assistant, the embeddings used by semantic search and recommendation, and the marketing drafts (phase 5). The embedding table already exists in the schema.
13	Maps / Geocoding (planned)	Will turn hotel addresses into coordinates and show hotels on a map. The hotel table already has latitude and longitude columns, but no provider has been chosen yet.
14	Social Media (planned)	Will publish approved marketing posts to the Facebook and Instagram pages of the hotel. The marketing screens exist on the web and mobile applications with sample data; the connection to the social platforms is not built yet.

Three data flows explain most of the design. In the booking flow the client creates a booking, the StayReco Service holds the nights in room_calendars for 15 minutes and returns the VietQR details; when SePay reports a transfer whose content holds the booking code and exactly the amount, the service confirms the booking, marks the invoice paid and credits the pending balance of the hotel wallet in one transaction. In the onboarding flow the partner uploads documents to Cloudinary through the StayReco Service, submits a verification request and a platform manager approves it, which makes the hotel visible in search. In the account flow every sign-up or invitation creates a one-time code or link that is sent by the Email Service and stored only as a hash.

A fourth flow, the recommendation flow, is planned for phase 5. The Recommendation Service runs on a schedule, reads the interaction data, trains the model and saves the results to the database. When a customer opens the home page or searches, the StayReco Service only reads those saved results, so a slow or failed training run never delays a request.

1.2 Technology Declaration

This section declares the third-party services, the development technologies with their versions, the source code and DevOps setup, the deployment environments and the AI tools used by the team, as required for Review 2.

1.2.1 Third-party Services

#	Service	Purpose	Data sent	Status
1	SePay	Payment: VietQR code and notification of incoming bank transfers	Booking code, amount and bank account in the QR; SePay sends the transaction id, amount, content and account number to our webhook	In use on staging

2	Cloudinary	Storage of hotel and room images and verification documents	Image and document files uploaded by partners	In use
3	Gmail SMTP (Nodemailer)	Authentication and transactional emails	Recipient name and email, one-time codes and links, booking code and amounts	In use
4	Render	Hosting of the API and the PostgreSQL database (staging)	All application data, as the hosting provider	In use
5	Vercel	Hosting of the web application (staging)	The built web application; pages call the API on Render and hold no database credentials	In use
6	GitHub	Source code hosting and CI (GitHub Actions)	Source code and CI logs	In use
7	OpenAI API	AI assistant, embeddings for semantic search and recommendation, marketing drafts	Questions of customers without contact details, hotel and room descriptions, marketing briefs	Planned (phase 5)
8	Firebase Cloud Messaging	Push notifications on mobile	Device token and notification text	Planned
9	Maps and geocoding provider (not chosen yet)	Convert hotel addresses into coordinates and show hotels on a map	Hotel address	Planned
10	Social media platforms (Facebook, Instagram)	Publish marketing posts approved by the hotel partner	Post text and images of the approved post	Planned

1.2.2 Development Technologies

#	Layer	Technology	Version
1	Front-end web	Next.js (App Router), React, TypeScript	Next.js 16.3.4, React 19.2.8, TypeScript 5
2	Front-end web	Tailwind CSS, shadcn/ui, TanStack Query, Zustand, Axios, React Hook Form, Recharts, i18next	Tailwind 4, TanStack Query 5, Zustand 5, Axios

			1.20, RHF 7, Recharts 3
3	Mobile	Expo, React Native, Expo Router, TanStack Query, Zustand, expo-secure-store	Expo SDK 57, React Native 0.86.3, React 19.2.3
4	Back-end	Node.js, NestJS, Prisma ORM with the pg adapter, Zod, JWT, argon2, Helmet, Throttler, Swagger	Node.js 22 (Render) and 20 (CI), NestJS 11, Prisma 7.10, Zod 4.6, argon2 0.45
5	Back-end	Nodemailer, Cloudinary SDK	Nodemailer 10, Cloudinary 2.11
6	Database	PostgreSQL with pgvector, pgcrypto and unaccent	PostgreSQL 16
7	Testing	Jest and Supertest (back-end unit and e2e tests), Playwright (web)	Jest 30, Supertest 7, Playwright 1.64

1.2.3 Source Code and DevOps

- Platform: one GitHub monorepo (KnightChovy/stay-reco) with the folders backend, frontend, mobile, packages/shared and docs. Each application is installed and run on its own.
- Branches: work is done on feature branches created from develop and merged into develop through pull requests. A release is a pull request from develop to main; main is the branch deployed to staging.
- Back-end CI (GitHub Actions, on every push and pull request to main and develop that touches the back-end or the shared package): install, ESLint, Prettier check, TypeScript check, unit tests with coverage, e2e tests against a pgvector PostgreSQL service container, and build.
- Front-end CI: install, ESLint, route type generation with TypeScript check, and production build of the web application.
- API changes follow the phase process in docs: the API page and the shared schemas are agreed first, then the back-end and front-end work in parallel on the same contract.

1.2.4 Deployment Environments

#	Environment	Where	How it is deployed and rolled back
1	Development	Each member's computer; PostgreSQL pgvector:pg16 in Docker Compose	npm run dev for the API and the web, expo start for mobile; prisma migrate dev creates migrations; npm run db:seed loads demo data.
2	Staging	Render, Singapore region: web service stayreco-staging-api (https://stayreco-staging-api.onrender.com) and PostgreSQL stayreco-staging-db	Automatic deploy on every change of main. The build runs npm ci and the build; the start command applies pending migrations (prisma migrate deploy) then starts the API. Render checks /api/v1/health. Rollback uses the Rollback action of Render to return to the previous deploy; migrations only move

			forward, so a schema fix is shipped as a new migration. Demo data is loaded with the seed script in the repo.
3	Web and mobile on staging	Web: Vercel, calling the staging API. Mobile: run with Expo against the staging API	Vercel builds and deploys the web application from the repository; a broken deploy is rolled back by promoting the previous deploy (Instant Rollback).
4	Production	Out of the capstone scope	The same setup on paid plans would be used; not planned during the project.

1.2.5 AI Tools

#	Tool	Model	Stages where it is used
1	Claude (Claude Code, Cowork)	Claude Opus	All five members use Claude, and the team uses no other AI assistant. It supports specification review and report drafting, design diagrams, code and tests, code review, and the self-review prompt of Review 2.

1.3 Package Diagram

1.3.1 Back-end

The back-end is a NestJS application organised by business module. Each module owns its controllers, services and DTOs; modules share the cross-cutting code in common and reach the database only through the Prisma service.

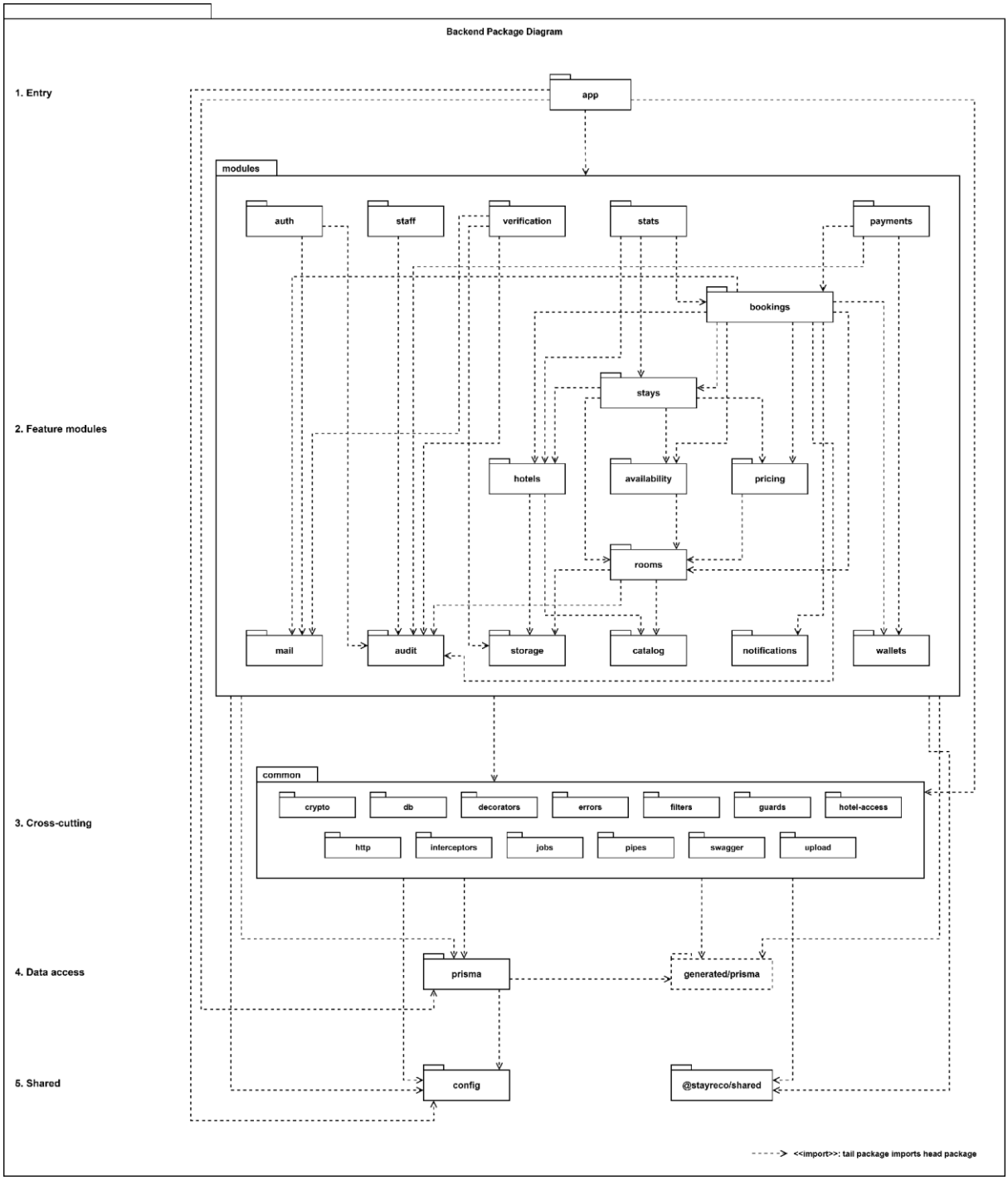


Figure 2. Back-end package diagram

#	Package	Description
1	main.ts, app.module.ts	Bootstraps the application, registers the global guards in order (rate limit, authentication, roles), Swagger and the modules.
2	config	Validates the environment variables at startup with zod; the API refuses to start with a wrong configuration.
3	common	Guards, decorators, the response interceptor, the exception filter, the zod validation pipe, error codes, Swagger helpers, field encryption,

		database locks, hotel access checks, upload helpers and the background job runner.
4	prisma	PrismaService on the pg driver adapter and the generated client.
5	auth	Registration, email code, login with lockout, sessions and refresh token rotation, password reset, invitations.
6	verification	Verification requests of hotels, documents and the decisions of platform managers.
7	hotels, rooms, catalog	Hotels, images, room types, physical rooms, amenities and bed types.
8	pricing, availability	Pricing rules and the price engine; the availability calendar and closing or reopening nights.
9	stays	Public search and hotel and room type pages with the price of the stay.
10	bookings	Booking with hold, my trips, voucher, cancellation, refund decisions, the front desk (check-in, check-out, no-show, rooms cleaned), change requests, settlement and the expiry and no-show jobs.
11	payments	SePay webhook, payment confirmation and the development payment simulator.
12	wallets	Wallet ledger: posts idempotent wallet transactions for hotels, customers and the platform; the hotel wallet history for the owner.
13	staff	Staff list, permission groups, deactivation and reactivation. The stats module computes the partner dashboard figures.
14	audit, mail, storage	Audit log writer, email sending with templates, Cloudinary storage. The notifications module stores the in-app notifications sent with each booking email.

1.3.2 Web Application

The web application is built with Next.js (App Router) and is organized into five layers, as shown in Figure 3. Routes in app are grouped by audience and only render feature screens. Features read server data through services using TanStack Query and keep client state in Zustand stores. Validation schemas and common types come from the shared package, which the mobile app and the back-end also use.

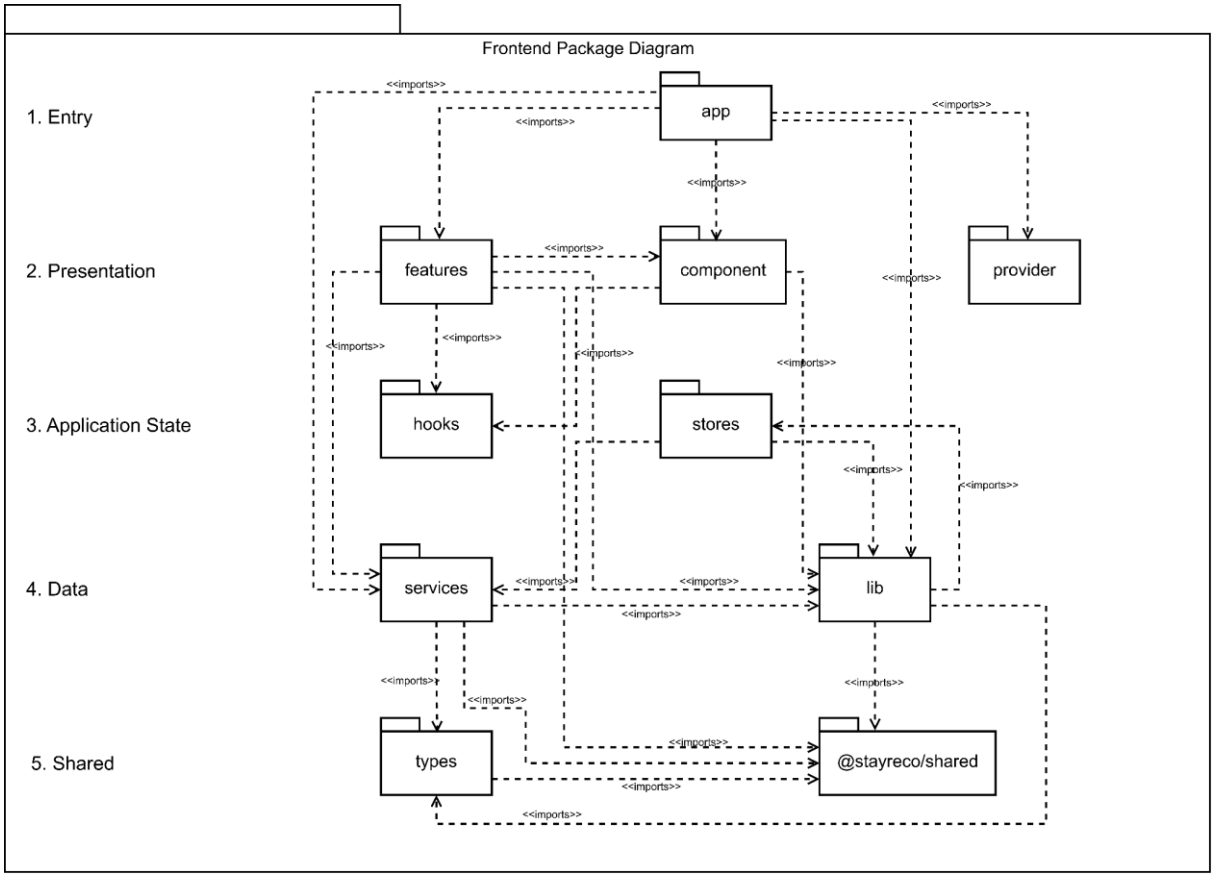


Figure 3. Web application package diagram

#	Package	Description
1	app	Routes grouped by audience: (landing-page) for public pages, (auth) for login and registration, search, hotel and booking pages for guests, and (actors) for the workspace of each role.
2	provider	Global providers wrapped around the app, mainly the TanStack Query client.
3	features	Screens and UI logic per business area: auth, booking, stays, account, partner, staff, manager and admin.
4	components	Reusable UI: ui holds the shadcn primitives, common holds shared cards, filters and tables, and layout holds the header and footer.
5	hooks	Small custom React hooks reused across features.
6	stores	Zustand stores for client state, mainly the signed-in user and auth tokens.
7	services	API call functions per area (auth, stays, partner, staff, admin), used by features through TanStack Query.
8	lib	Axios instance with token refresh, API error mapping and utility helpers.
9	types	TypeScript types used only inside the web application.

10	@stayreco/shared	Workspace package with the Zod schemas and types shared by the web, mobile and back-end.

1.3.3 Mobile Application

The mobile application is built with React Native (Expo) and Expo Router and follows the same layered structure as the web application (Figure 4). Customers use a tab layout, while hotel staff have a separate route group for front-desk operations. Input validation reuses the Zod schemas from the shared package, so the mobile app, the web and the API apply the same rules.

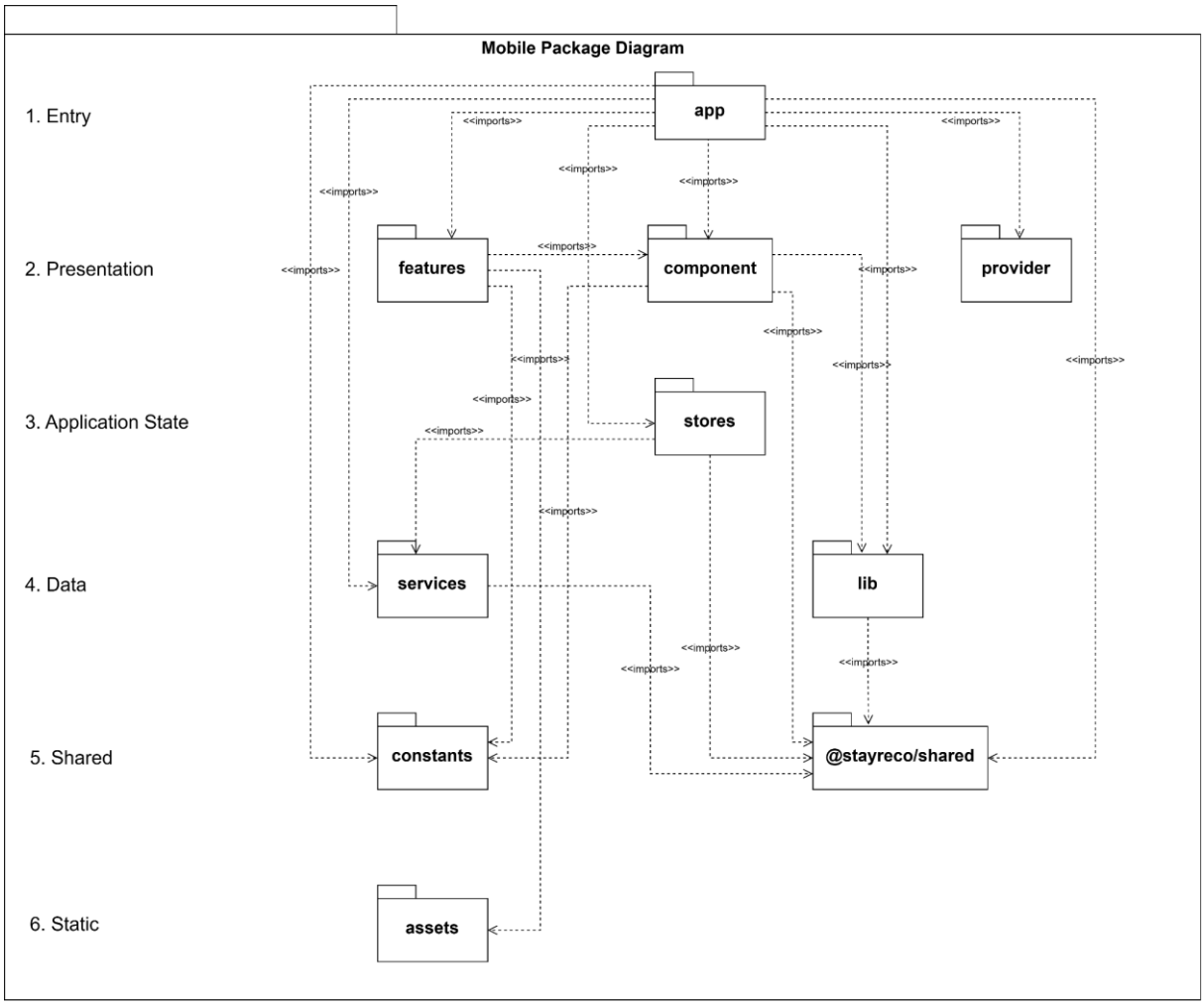


Figure 4. Mobile application package diagram

#	Package	Description
1	app	Expo Router routes: (tabs) for home, search, trips, assistant and account; (auth) for login, registration and password recovery; screens for hotel details, booking, payment, confirmation and notifications; a staff group for bookings, check-in, check-out, refunds, alerts, chat and marketing.
2	provider	Root providers, mainly the TanStack Query client and the theme.

3	features	Feature-specific UI and logic, such as booking progress, price lines and social login actions.
4	components	Shared UI elements: buttons, inputs, chips, QR code and cards.
5	stores	Auth store; tokens are persisted with expo-secure-store.
6	services	ApiService handles requests and tokens; domain services cover auth, stays, bookings, notifications and staff.
7	lib	Utility helpers used across the app.
8	constants	Theme values and app-wide constants.
9	@stayreco/shared	Zod schemas and types shared with the web and the back-end.
10	assets	Images, icons and fonts bundled with the app.

2. Database Design

2.1 Logical ERD

The logical model refines the conceptual ERD of Report 3 into 56 tables. Every table has a UUID primary key generated by the database, status and type columns use PostgreSQL enums, money uses DECIMAL(14,2) in VND, and time uses timestamptz. Accounts follow class table inheritance: user_account holds the common data and each of the five roles has its own table whose primary key is also a foreign key to user_account. To keep each diagram readable, the model is split into seven business areas. In each figure the tables of the area are shown with all their columns; tables of other areas that they reference are shown in grey with their key only.

Notation: PK is the primary key, FK a foreign key and U a unique column; a bullet marks a required column. Relationships use crow's foot notation.

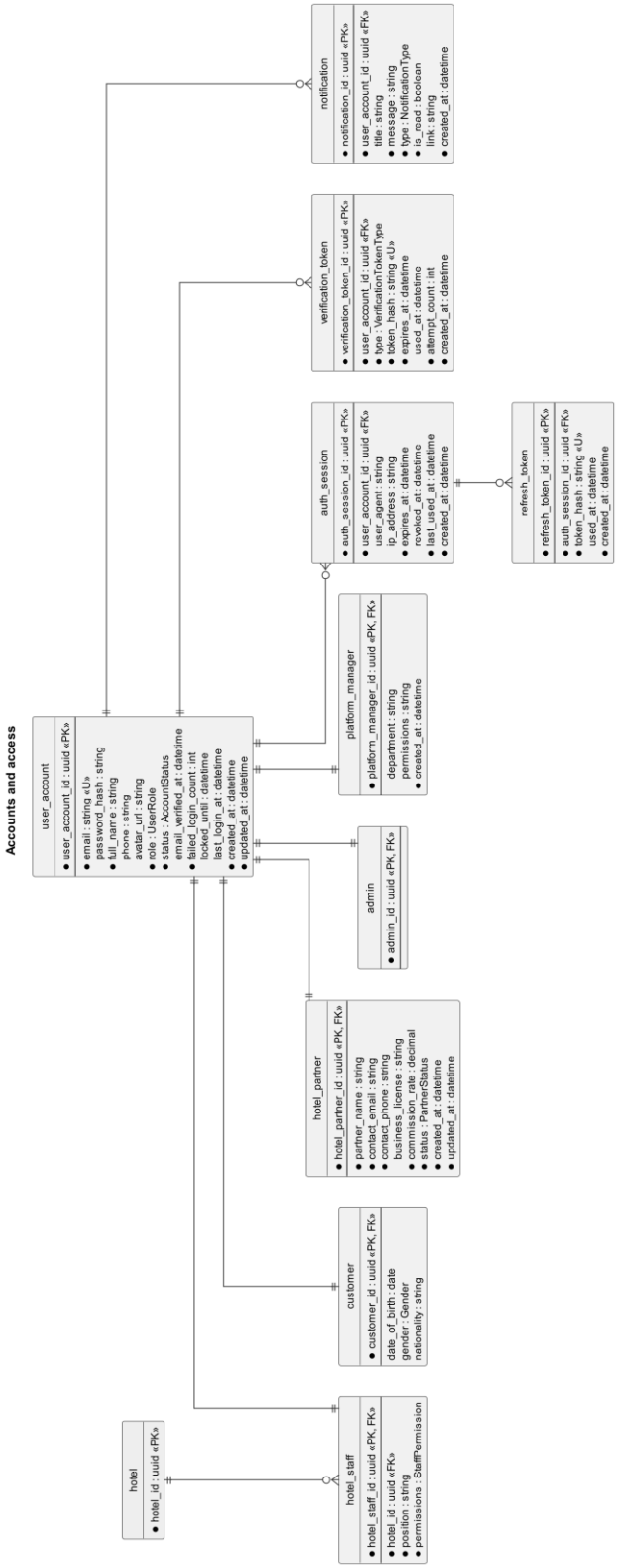


Figure 5. Logical ERD: accounts and access

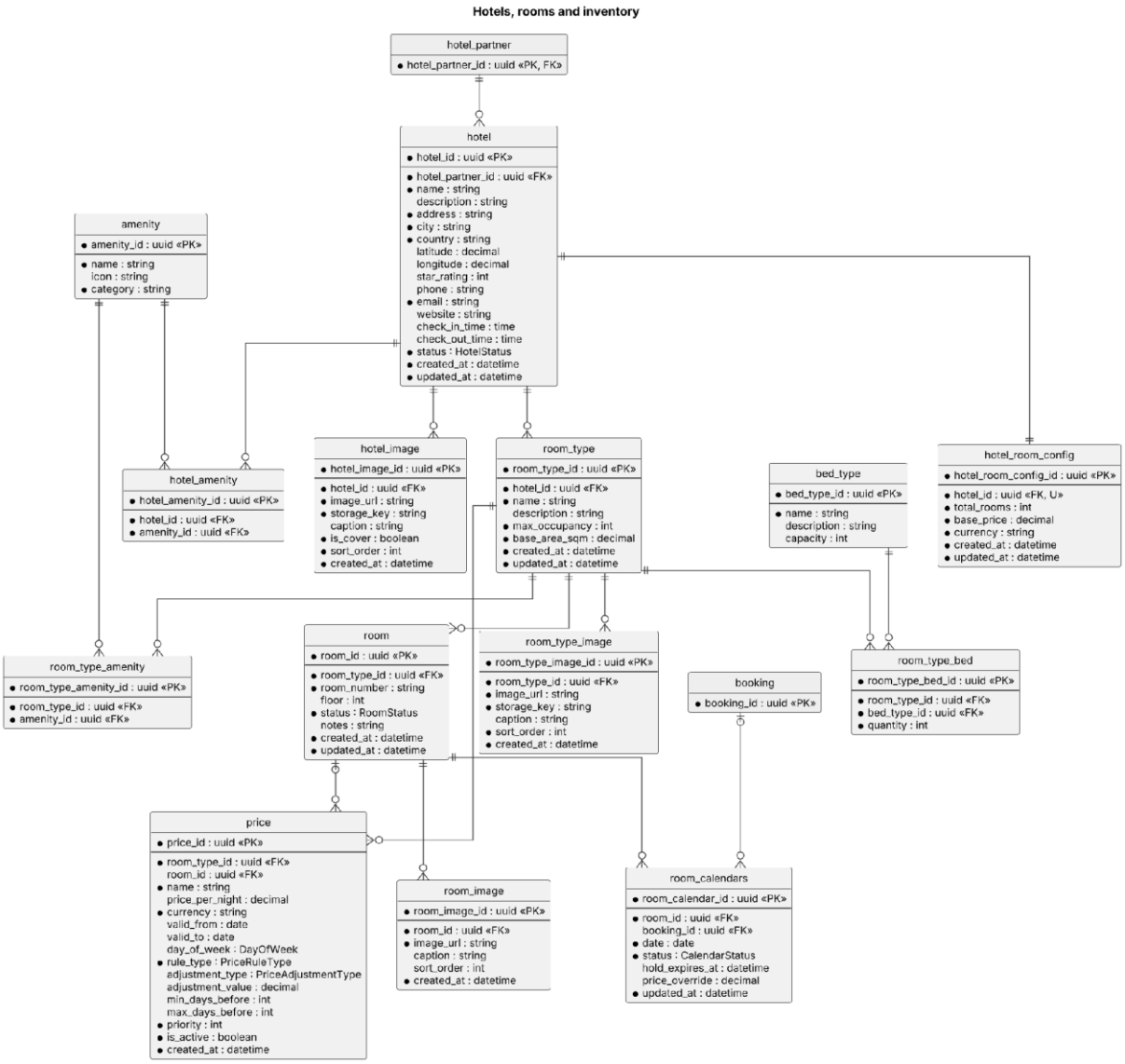


Figure 6. Logical ERD: hotels, rooms and inventory

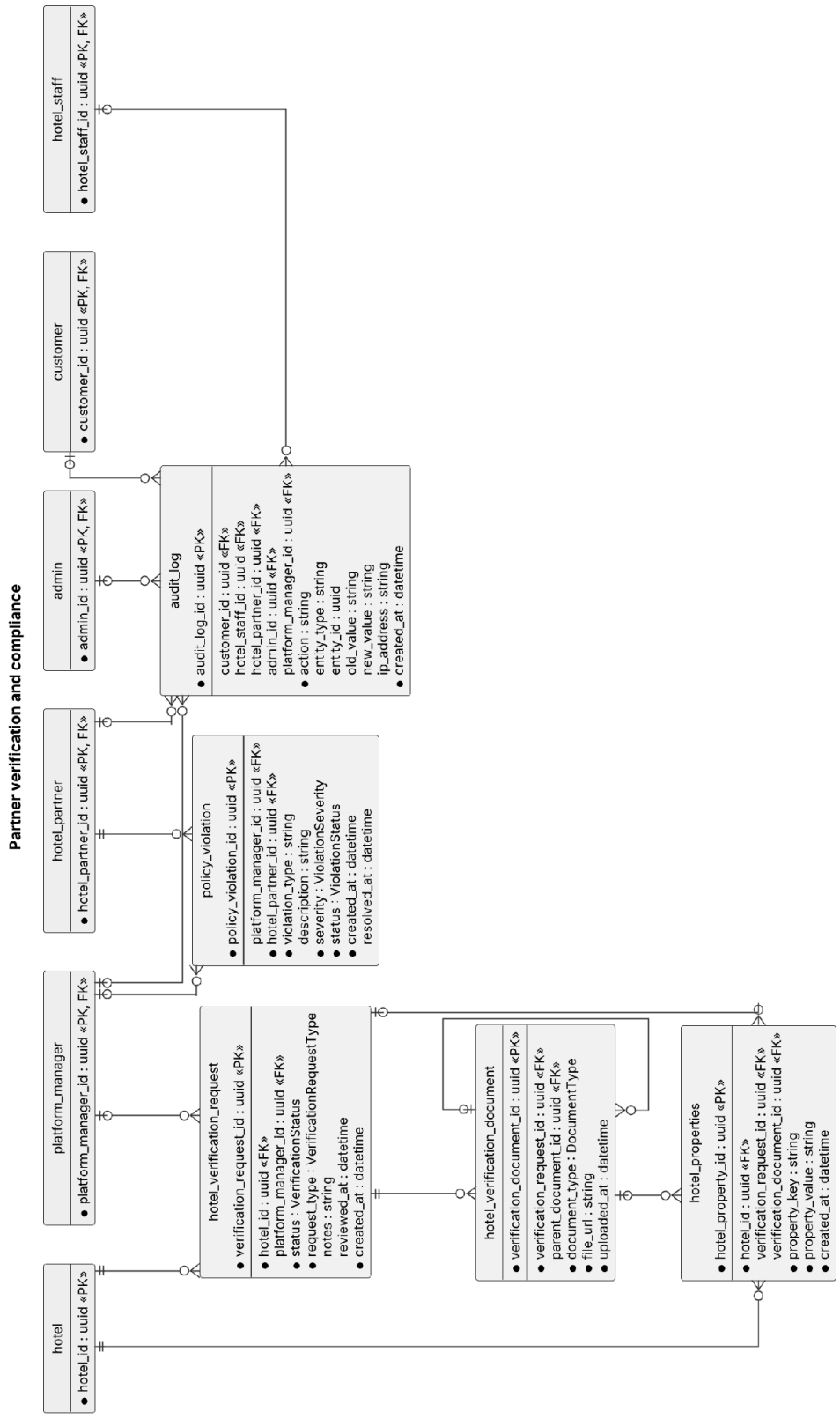


Figure 7. Logical ERD: partner verification and compliance

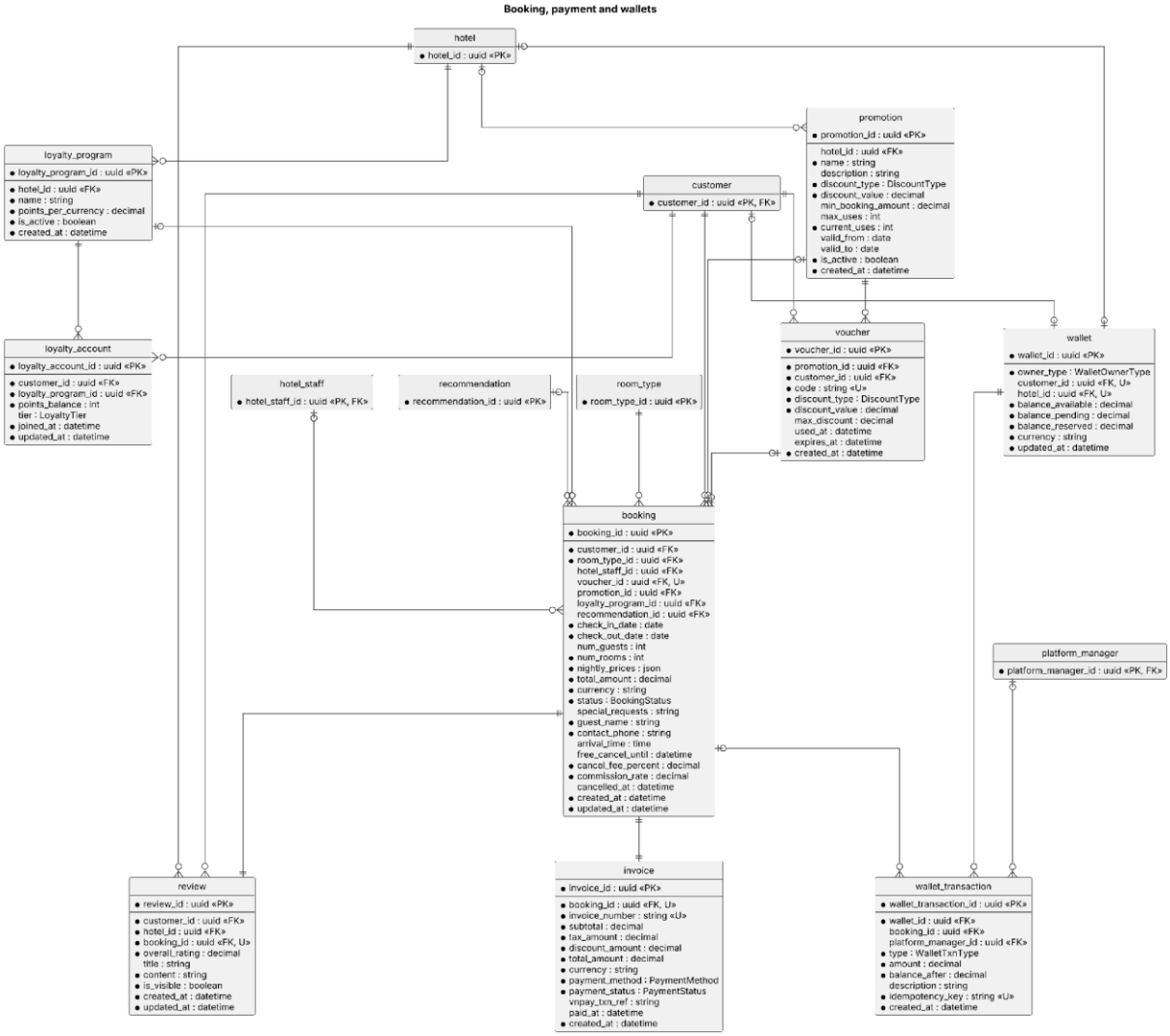


Figure 8. Logical ERD: booking, payment and wallets

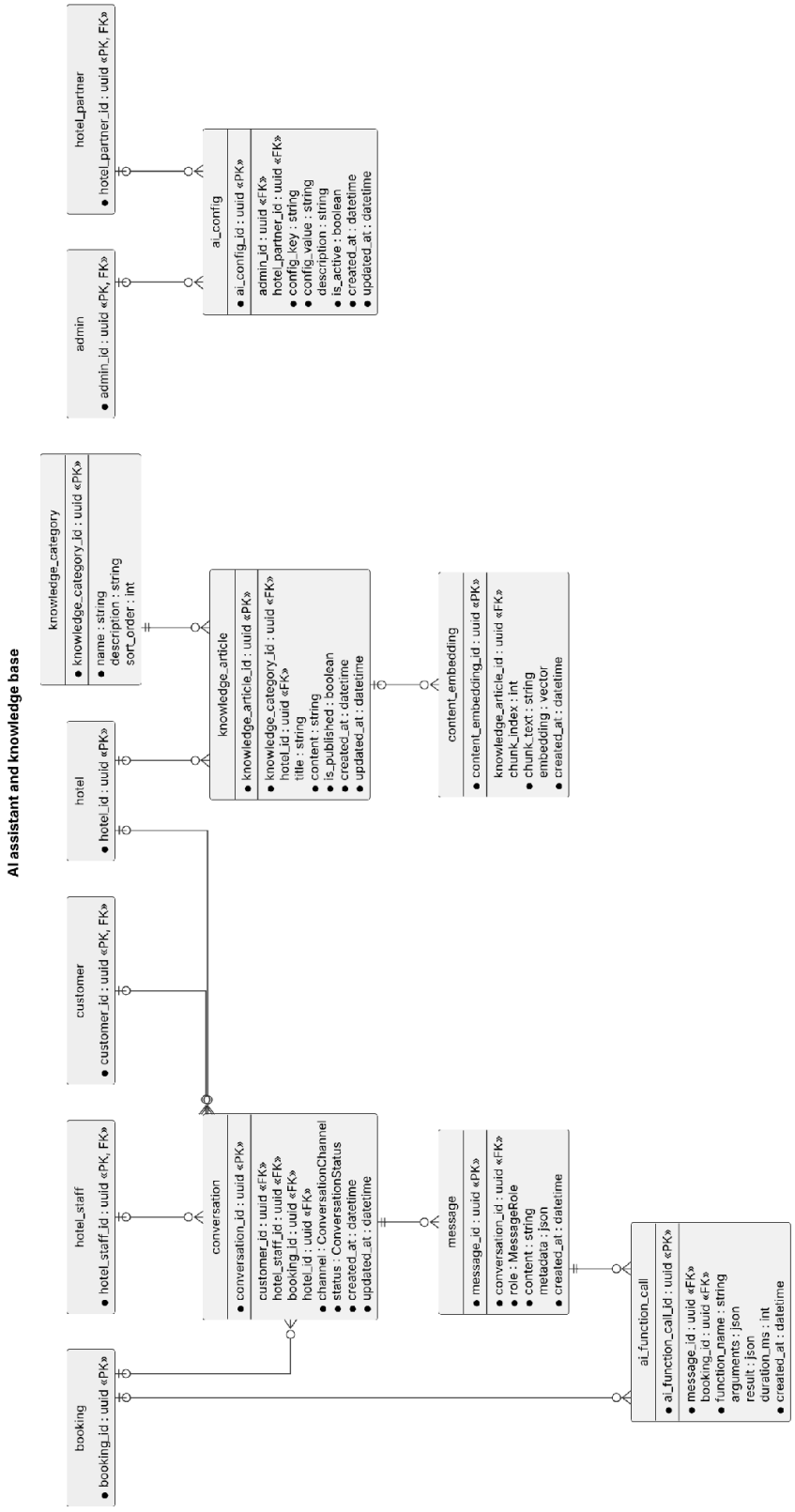


Figure 9. Logical ERD: AI assistant and knowledge base

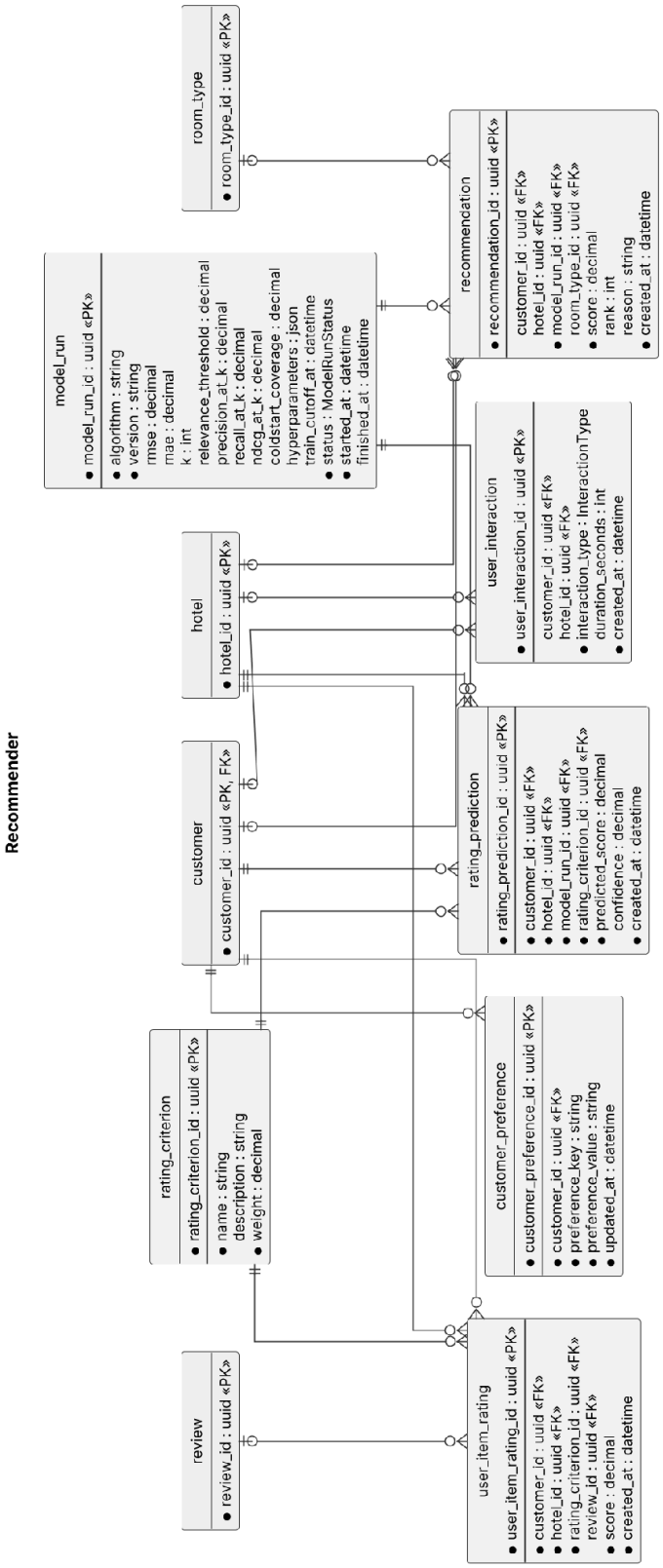


Figure 10. Logical ERD: recommender

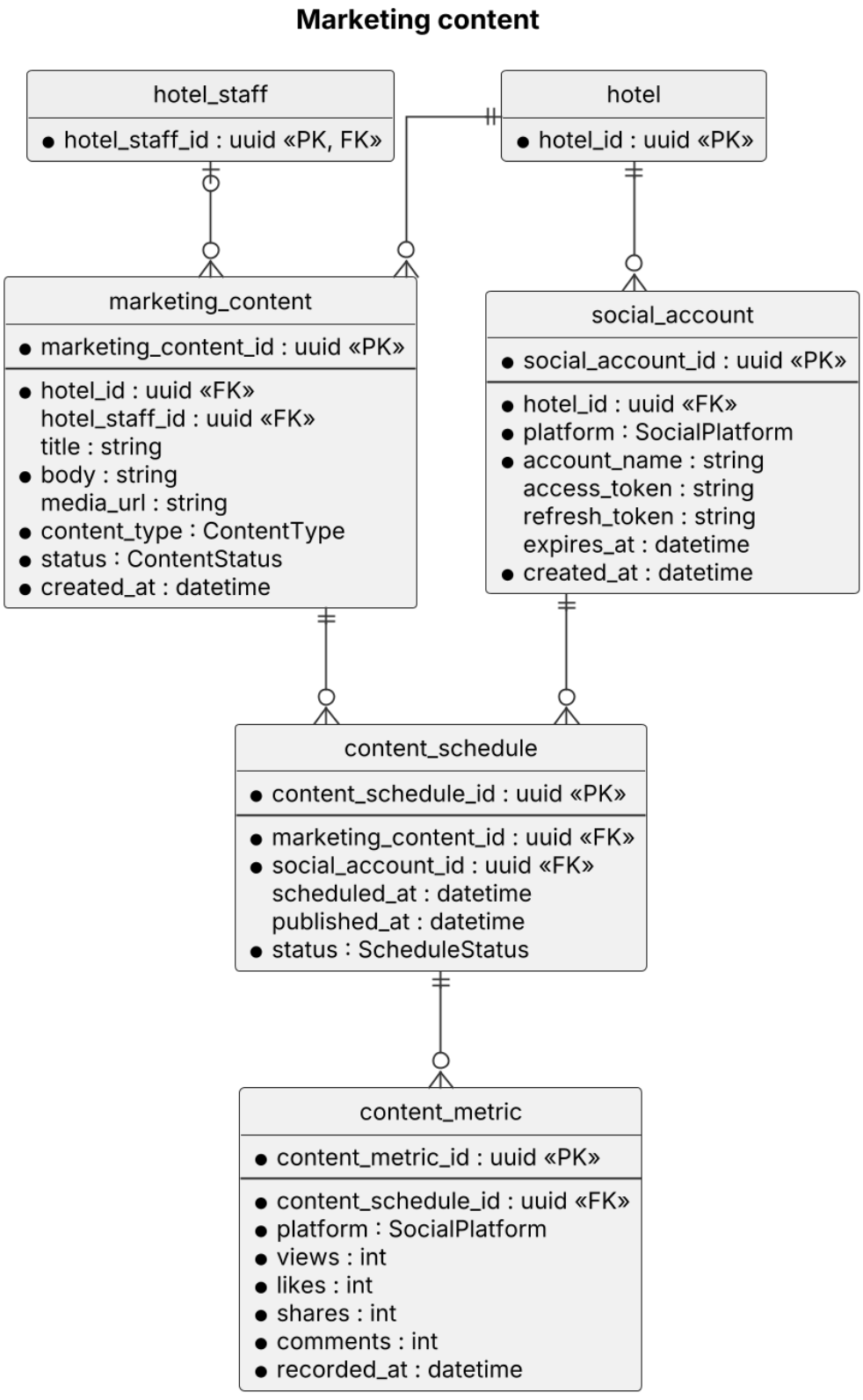


Figure 11. Logical ERD: marketing content

2.2 Table Descriptions

#	Table	Description

1	user_account	Every account in the system with email, password hash, role and status, plus the login lock counters. Role-specific data lives in the five role tables below.
2	customer	Traveller profile (date of birth, gender, nationality). Its key is also the user_account key.
3	hotel_partner	Hotel owner: business name, contact, commission rate and partner status. One partner can own several hotels.
4	hotel_staff	Staff account bound to one hotel, with a position and the permission groups granted by the partner.
5	admin	System administrator account.
6	platform_manager	Platform manager account (department and permissions) who verifies partners and follows revenue.
7	auth_session	One signed-in device: user agent, IP, expiry and revocation time.
8	refresh_token	Hashed refresh tokens of a session; a used token cannot be used again (rotation).
9	verification_token	Hashed one-time email code, password reset link or invitation link, with expiry and wrong-attempt count.
10	notification	In-app notification of an account (booking, payment, review, alert, system).
11	hotel	A hotel of a partner: address, location, star rating, check-in and check-out times, and listing status.
12	hotel_image	Ordered photos of a hotel stored on Cloudinary, one of them marked as cover.
13	hotel_amenity	Amenities offered by a hotel (link table).
14	amenity	Catalog of amenities with icon and category, shared by hotels and room types.
15	hotel_room_config	Total number of rooms and default base price of a hotel, filled in the onboarding wizard.
16	room_type	Room type of a hotel: name, description, maximum occupancy and area.
17	room_type_image	Ordered photos of a room type.
18	room_type_amenity	Amenities of a room type (link table).
19	room_type_bed	Beds of a room type with their quantity.
20	bed_type	Catalog of bed types and their capacity.

21	room	Physical room of a room type with room number, floor and housekeeping status.
22	room_image	Photos of one physical room.
23	room_calendars	One room on one night: held, booked or blocked, with the booking and the hold expiry. Unique per room and date, which prevents overbooking.
24	price	Pricing rules of a room type: base price, seasonal, early-bird and last-minute adjustments with priority and validity.
25	hotel_verification_request	One verification request of a hotel (initial, update or renewal), its status and the reviewing manager.
26	hotel_verification_document	Documents attached to a request (business license, fire safety, ID card...), with a link to the replaced document.
27	hotel_properties	Key-value snapshot of the legal entity and beneficiary data submitted with a request; the bank account number is encrypted.
28	policy_violation	Compliance case opened by a platform manager against a partner, with severity and status.
29	audit_log	Append-only trail of important actions (who, what, old and new value, IP) and of SePay transfers kept for manual handling.
30	booking	A reservation of one or more rooms of one room type: stay dates, nightly prices fixed at booking time, total, cancellation terms, commission rate and status.
31	invoice	Payment record of a booking; the invoice number is the booking code used as the transfer content. Holds the payment status and the SePay transaction id.
32	wallet	Wallet of a customer, a hotel or the platform, with available, pending and reserved balances.
33	wallet_transaction	Every money movement of a wallet (payment, commission, refund, payout...), with the balance after it and a unique idempotency key.
34	promotion	Discount campaign of a hotel with validity, minimum amount and usage limit.
35	voucher	Discount code issued to a customer from a promotion, used by at most one booking.
36	loyalty_program	Loyalty program of a hotel (points earned per amount spent).
37	loyalty_account	Points balance and tier of a customer in one hotel program.

38	review	Review of a completed stay (one per booking) with the overall rating and the visibility flag.
39	conversation	Chat thread of a customer with the AI assistant or hotel staff, on web or mobile, optionally linked to a booking or hotel.
40	message	Messages of a conversation with their role (user, assistant, staff, system).
41	ai_function_call	Tool calls made by the assistant for a message (name, arguments, result, duration).
42	ai_config	AI settings at platform level (admin) or brand level (partner) as key-value pairs.
43	knowledge_category	Category of the knowledge base used by the assistant.
44	knowledge_article	Article of the knowledge base, platform-wide or for one hotel.
45	content_embedding	Text chunks of an article with their pgvector embedding for semantic search.
46	customer_preference	Travel preferences declared by a customer (key-value).
47	user_interaction	Implicit feedback of a customer on hotels (view, search, click, wishlist, book).
48	user_item_rating	Explicit rating of a customer for a hotel on one criterion, taken from a review.
49	rating_criterion	The rating criteria (for example cleanliness, location, service) and their weight.
50	rating_prediction	Predicted score of a customer for a hotel on one criterion, produced by a model run.
51	recommendation	Ranked hotel or room type suggestion for a customer with its score and reason.
52	model_run	One training run of the recommender with its algorithm, hyper-parameters and metrics (RMSE, MAE, Precision@K, Recall@K, NDCG@K).
53	marketing_content	Marketing post, blog article or caption of a hotel written with AI, with its approval status.
54	content_schedule	Publishing slot of a content on one social account.
55	content_metric	Views, likes, shares and comments collected for a published post.
56	social_account	Social media account of a hotel (Facebook, Instagram, TikTok) with its access tokens.

2.3 Database Script

The database script is docs/db/stayreco_schema.sql in the repository (tag review-2). It is the concatenation, in order, of the fourteen Prisma migrations in backend/prisma/migrations, so it always matches the tables of the logical ERD: running it on an empty PostgreSQL 16 database creates the three extensions, the 34 enums, the 56 tables with their keys, unique constraints, check constraints and indexes. In practice the team applies the same migrations with prisma migrate deploy, which is also what the staging deploy runs.

- Demo data: npm run db:seed runs backend/prisma/seed.ts (amenities, bed types, the admin account) and seed-demo.ts (demo hotels, three active and one waiting for verification, with their partners, staff, room types, rooms and prices, plus a platform manager), so the demo on staging can be repeated.
- Rules enforced by the database: one row per room and night in room_calendars (no overbooking), a unique idempotency key per wallet transaction (no double payment), one invoice and at most one review per booking, unique booking code, and unique email per account.
- Sensitive data: passwords use argon2 hashes, one-time codes and refresh tokens are stored as hashes, and the beneficiary bank account number is encrypted with AES-256-GCM before it is written.

3. Detailed Design

The detailed design gives, for each main feature, the class diagram of the classes involved and the sequence diagrams of its main operations. The features below follow the five main workflows; each one is written by the owner of the workflow, and the diagrams are kept as PlantUML sources in docs/diagrams of the repository.

#	Feature	Sequence diagrams	Owner
1	Authentication	Register with email code, login, refresh token, reset password	Dinh Hoang Phuc
2	Partner onboarding and verification	Submit verification, review request	Dinh Hoang Phuc
3	Search and booking	Search stays, create booking with hold	Nguyen Kim Thien Ngan
4	Payment	Show VietQR, SePay webhook, hold expiry	Nguyen Kim Thien Ngan
5	Recommendation	Get recommendations	Ngo Minh Tan, Dinh Hoang Phuc
6	Check-in, check-out and review	Check in with QR, check out, write review	Nguyen Binh Khiem
7	Cancellation and refund	Cancel booking, forward refund, approve or reject refund, automatic approval	Phan Tan Phat
8	AI blog	Generate draft, submit and approve, publish	Ngo Minh Tan

Sections 3.1 to 3.8 will be added by the owners before the Review 2 submission.

4. Business Flows and Entity States

4.1 Activity Diagrams

Each main workflow has one activity diagram with one swimlane per actor. The diagrams show business steps only, not the processing inside the application, and end with the business result and the state changes of the main entities. Each diagram was drawn by the owner of the workflow; the draw.io sources are in docs/diagrams/activity. Dashed grey elements are planned and not implemented yet.

#	Workflow	Actors	Owner
1	WF-01 Partner onboarding and hotel verification	Hotel Partner, Platform Manager	Dinh Hoang Phuc
2	WF-02 Search, book and pay	Customer, SePay (AI recommender)	Nguyen Kim Thien Ngan, with Dinh Hoang Phuc and Ngo Minh Tan for the recommendation
3	WF-03 Check-in, check-out and review	Customer, Hotel Staff	Nguyen Binh Khiem
4	WF-04 Cancellation and refund	Customer, Hotel Staff, Hotel Partner	Phan Tan Phat
5	WF-05 AI-assisted blog post	Hotel Staff, AI service, Hotel Partner	Ngo Minh Tan

4.1.1 WF-01 Partner Onboarding and Hotel Verification

The partner signs up and confirms the email with a 6-digit code. The code lasts 10 minutes and allows five attempts; a new code can be sent at any time and the old one stops working. After activation the partner creates the hotel as a draft, sets up the rooms and fills the four-step verification wizard. Documents are stored privately on Cloudinary and the bank account is encrypted.

The system checks the request before accepting it. A Platform Manager then reviews the documents through a link that lasts five minutes and approves, rejects or asks for changes. Approval makes the hotel ACTIVE; rejection or a change request sends the hotel back to DRAFT and emails the reason so the partner can revise and submit again.

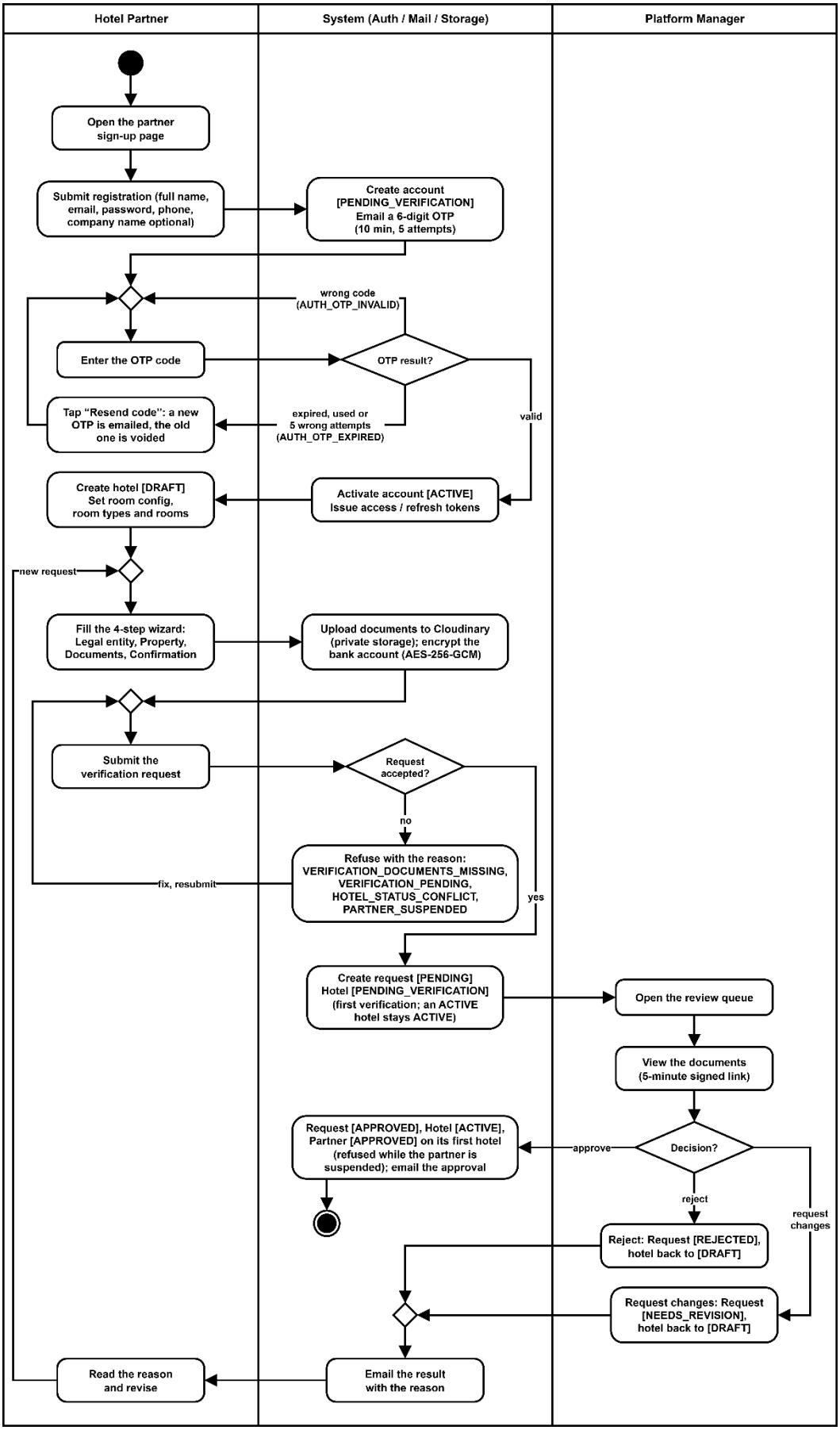


Figure 12. Activity diagram of WF-01 Partner Onboarding and Hotel Verification

4.1.2 WF-02 Search, Book and Pay

The customer searches by destination, dates, guests and rooms. The system normalises the destination, keeps only hotels that are on sale and shows each result with the total price of the stay. AI recommendations are drawn in dashed grey because they are planned.

When the customer books, the system prices the stay again and checks every night. If the price changed or a room is sold out, the customer is told and chooses again. Otherwise the rooms are held for 15 minutes and an unpaid invoice is created. The customer pays by VietQR; the SePay webhook confirms the booking, marks the invoice as paid and adds the amount to the hotel wallet as pending. A booking still unpaid after 15 minutes expires and the rooms are released. A late or unmatched transfer is written to the audit log for manual handling.

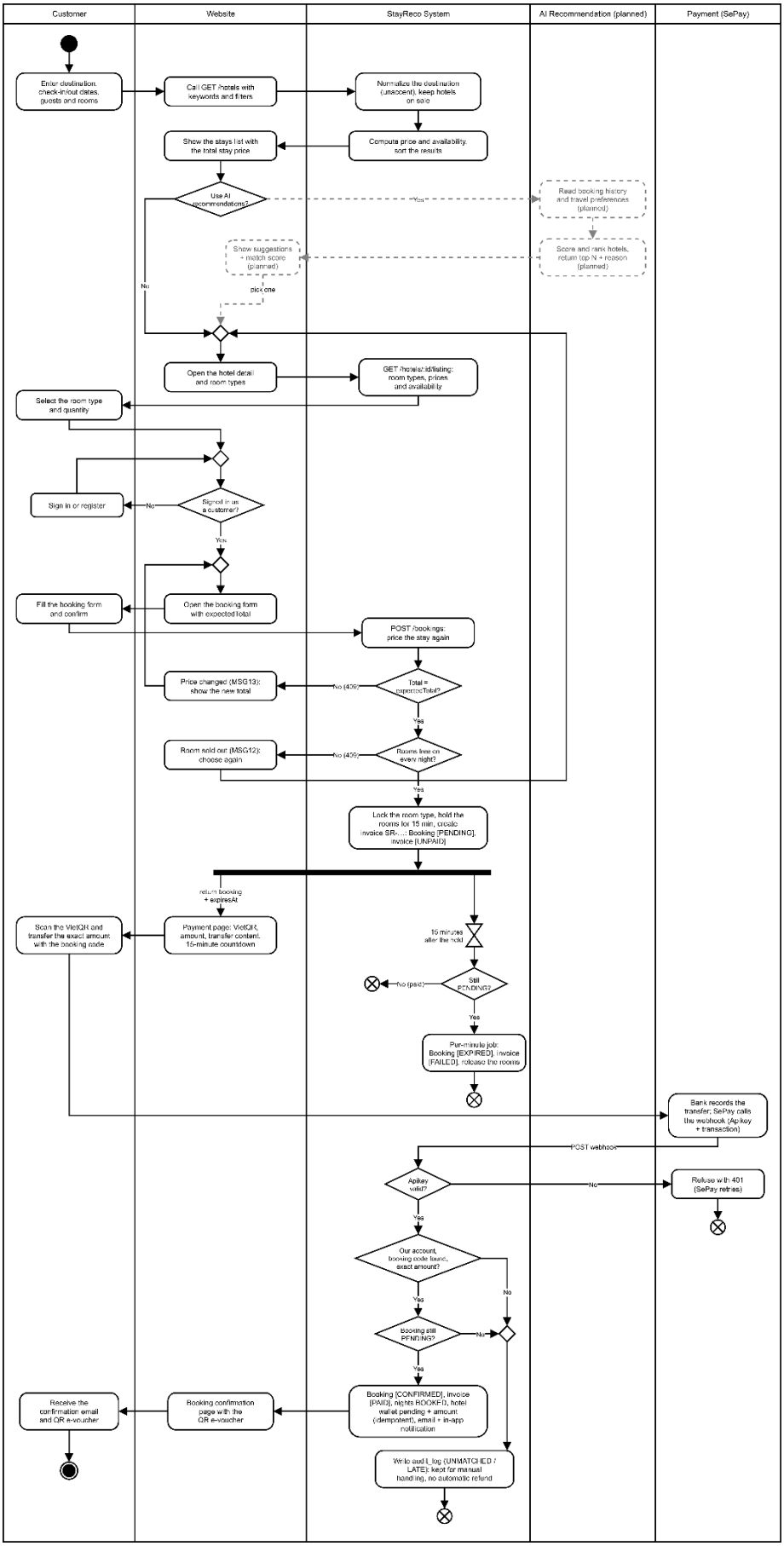


Figure 13. Activity diagram of WF-02 Search, Book and Pay

4.1.3 WF-03 Check-in, Check-out and Review

At the desk the staff scan the QR voucher or type the booking code. Check-in is refused if the booking is not confirmed and paid, a refund decision is still waiting, or today is outside the stay dates. Staff keep the held rooms or move the guest to other free rooms of the same type; the booking becomes CHECKED_IN and the rooms OCCUPIED. A guest who does not arrive is marked NO_SHOW and the hotel keeps the payment.

At check-out the system quotes the late fee from the hotel policy. Staff may lower the fee with a reason but never raise it. The booking becomes COMPLETED, unused nights are released without a refund, the commission is charged and the room goes to cleaning. One day later the hotel net amount moves from pending to available. Writing a review after the stay is planned.

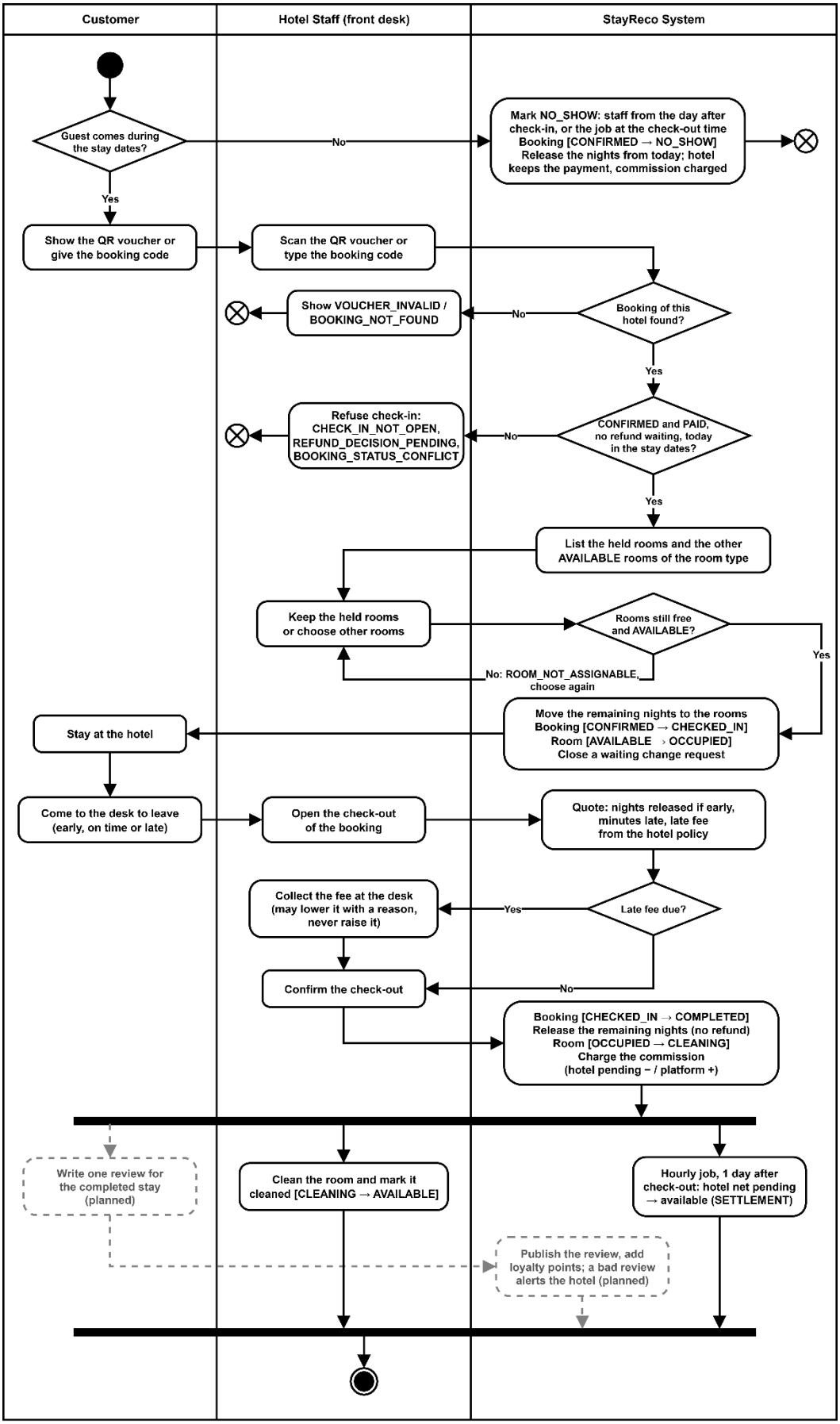


Figure 14. Activity diagram of WF-03 Check-in, Check-out and Review

4.1.4 WF-04 Cancellation and Refund

The customer opens the booking and sees the cancellation fee and the refund amount before confirming. The fee is zero before the free-cancellation deadline and a fixed percentage of the amount paid after it; both values are fixed on the booking when it is made. No cancellation is possible after check-in.

If the refund is zero, the booking is cancelled at once and the hotel keeps the full amount. Otherwise the invoice moves to REFUND_PENDING while the rooms stay held. Hotel staff can forward the request to the owner with a note, and the owner approves or rejects it with a reason. A request still pending at the check-in time is approved automatically. On approval the refund goes to the customer's StayReco wallet and the hotel wallet is reduced; a kept fee is settled to the hotel one day later.

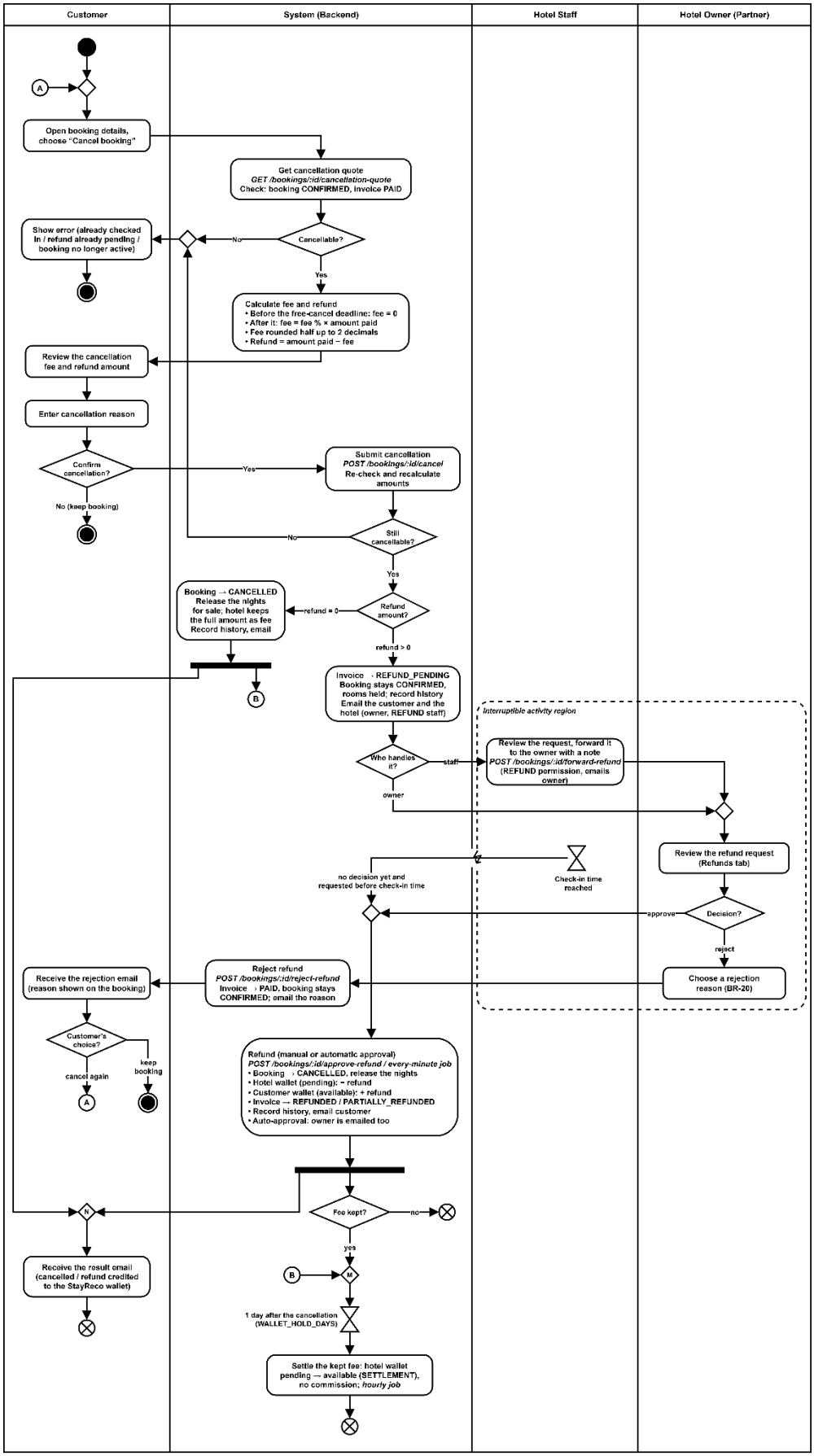


Figure 15. Activity diagram of WF-04 Cancellation and Refund

4.1.5 WF-05 AI-assisted Blog Post

Hotel staff write a blog post with AI help or by hand. With AI, the service reads the hotel brand settings, generates three options and filters any option that breaks the guardrails. The staff pick one, edit it and save it as a draft.

The draft is then sent to the hotel partner for review. The partner either approves it or rejects it with feedback, and a rejected post goes back to draft for revision. An approved post is published on the hotel blog at once or scheduled for a later time.

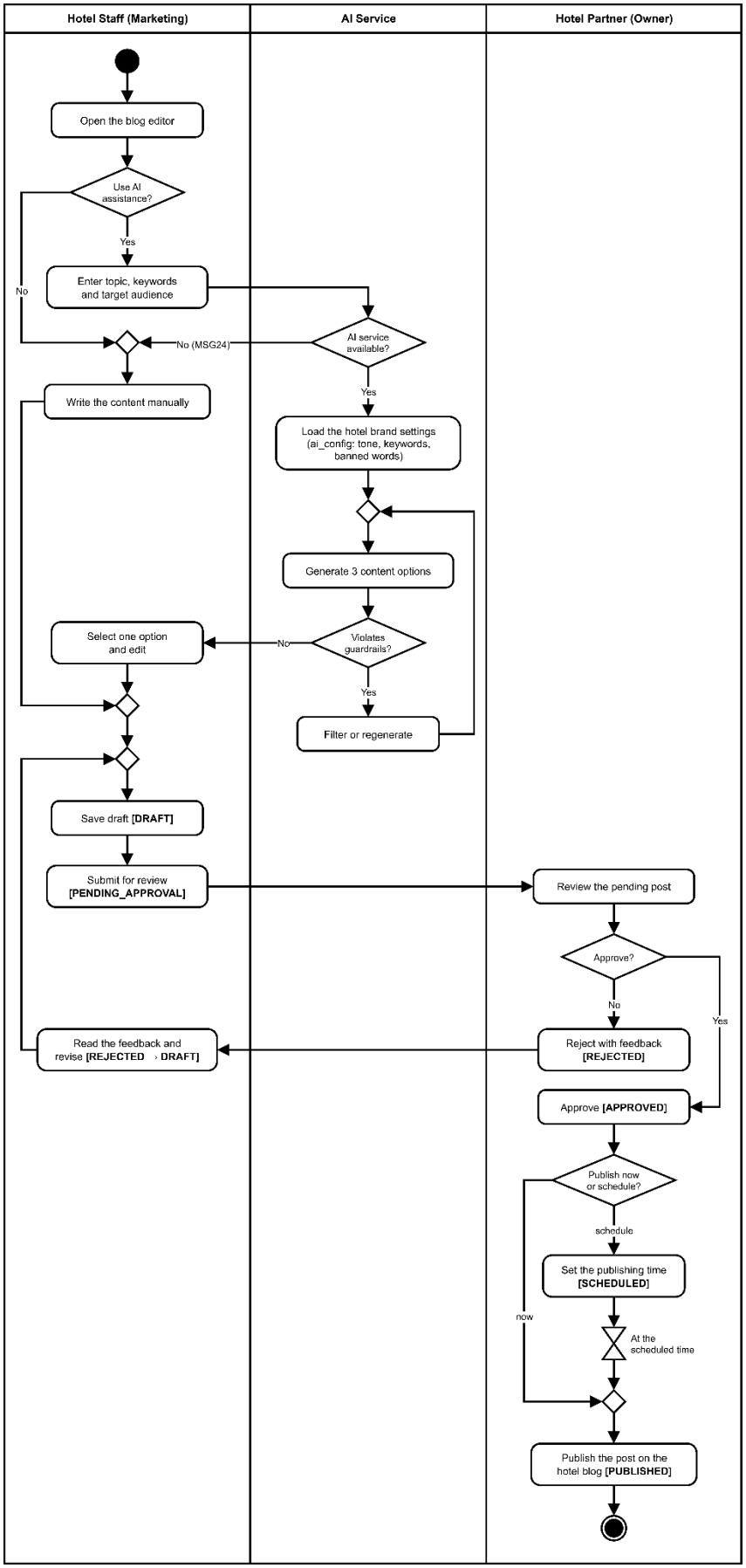


Figure 16. Activity diagram of WF-05 AI-assisted Blog Post

4.2 State Machine Diagrams

This section shows the life cycle of each main entity whose status changes over time. States and transitions follow the implementation at the review-2 tag. Solid arrows are implemented transitions; dashed arrows and transitions marked (planned) belong to later phases.

4.2.1 Account

An account starts in one of two ways. Customers and partners who register wait for email verification and become active after entering the correct code. Staff and platform managers are invited; the account becomes active when they accept the invitation and set a password, and the flow ends if the partner cancels the invitation. A partner can deactivate and reactivate a staff account. Suspension and deletion by the admin are planned.

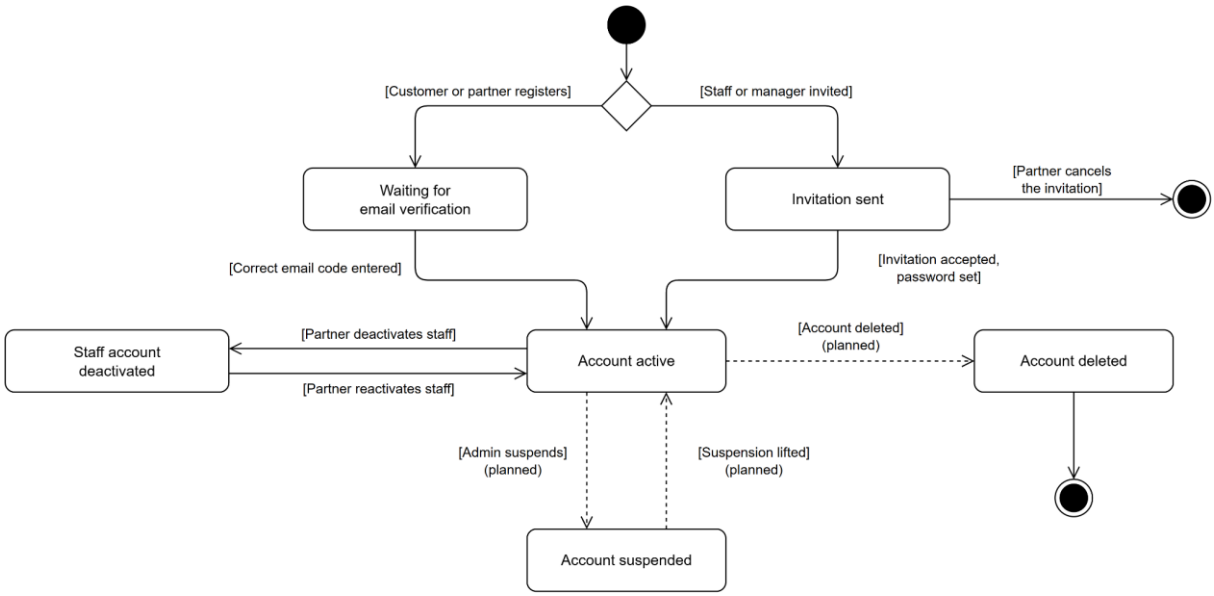


Figure 17. State machine diagram of account

4.2.2 Verification token

Every email code or link is issued once and ends in exactly one of four states. It is used when the correct code is entered or the link is opened, expires when its time limit is reached (10 minutes for a code, 15 minutes for a password reset, 72 hours for an invitation), is cancelled after the fifth wrong code, or is replaced when a new code or link is sent.

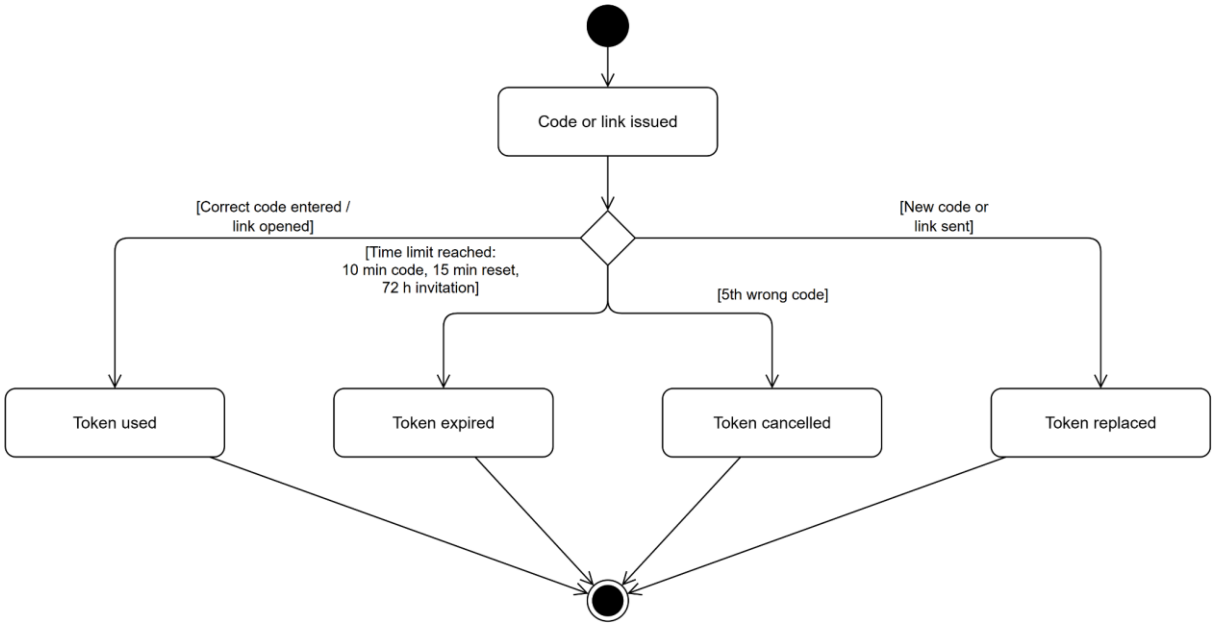


Figure 18. State machine diagram of verification token

4.2.3 Hotel

A hotel starts as a draft when the partner adds it and waits for verification once the partner submits the first request. It becomes active when a manager approves the request and the partner is not suspended, and goes back to draft when the manager rejects the request or asks for changes. Suspension by a manager after a violation, temporary deactivation by the partner and closing a hotel for good are planned.

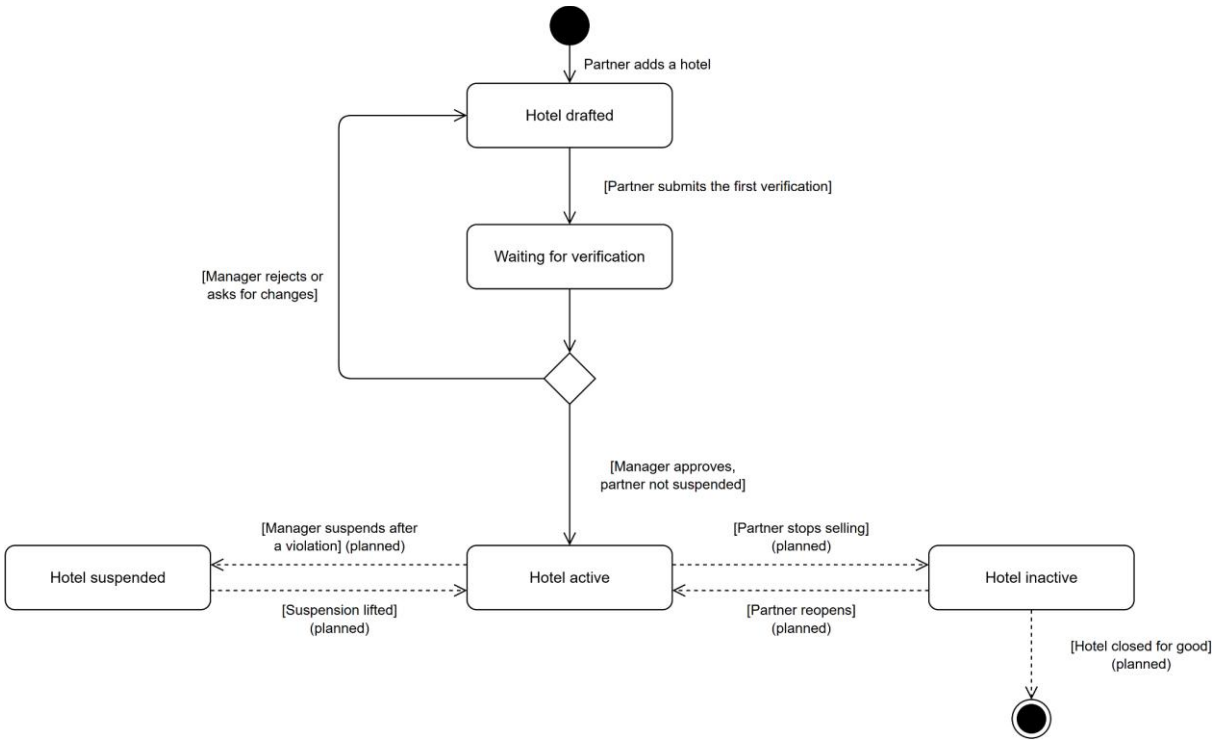


Figure 19. State machine diagram of hotel

4.2.4 Verification request

A verification request is created in one of two ways. When the partner submits an INITIAL or UPDATE request, it waits for review and the manager approves it, rejects it with a reason, or asks for changes. When a manager asks an active hotel to verify again, a RENEWAL request is created directly in the revision-needed state. Each request is decided only once and then ends.

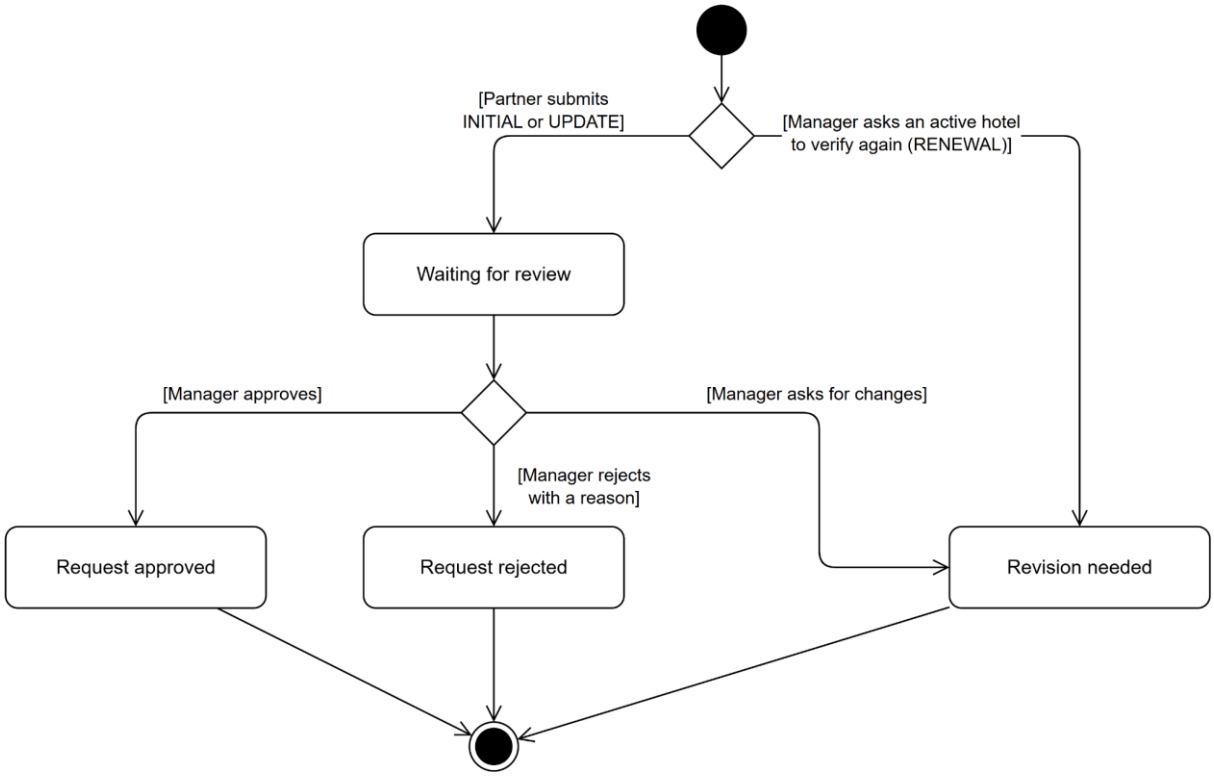


Figure 20. State machine diagram of verification request

4.2.5 Physical room

A room is available as soon as the partner adds it. Check-in makes the assigned room occupied, check-out moves it to cleaning, and staff mark it available again once it is cleaned. The hotel can put an available or cleaning room under maintenance and bring it back when maintenance is finished; an occupied room cannot be put under maintenance. Removing a room is planned.

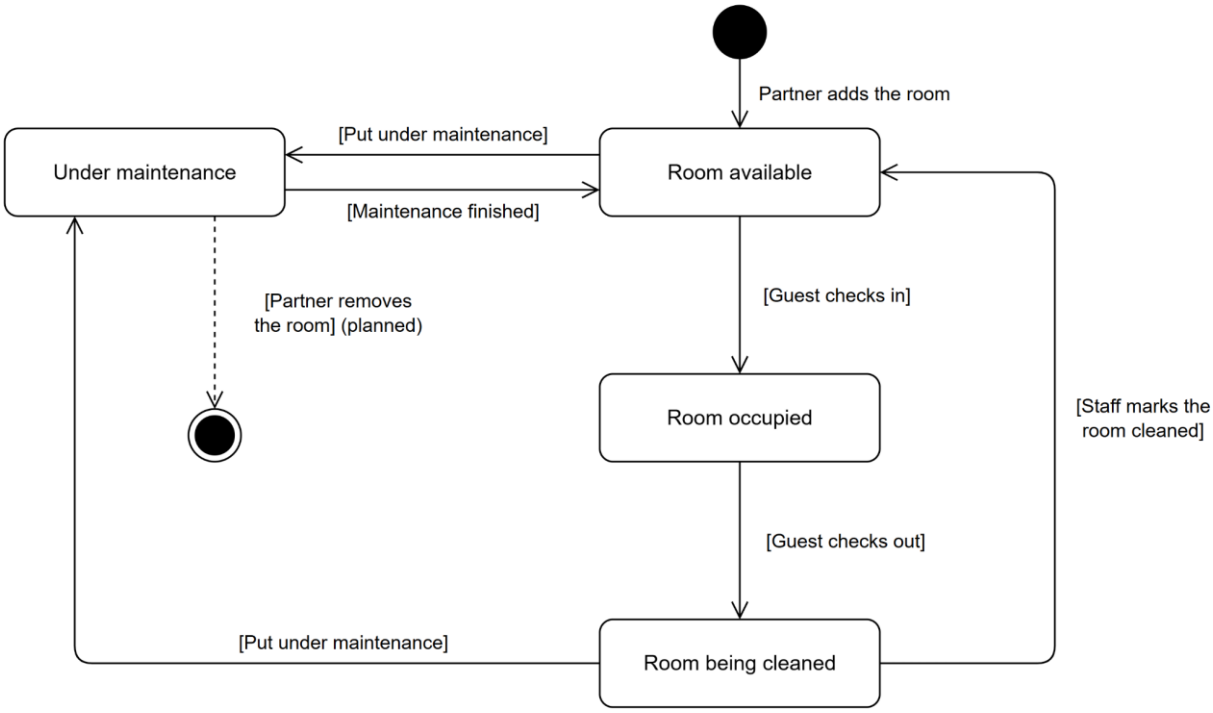


Figure 21. State machine diagram of physical room

4.2.6 Room night

This diagram shows the inventory of one room for one night. A night on sale is held for 15 minutes when a customer books, goes back on sale if the hold expires, and becomes booked when payment is confirmed. The partner can also close a night and reopen it later. A booked night goes back on sale when the booking is cancelled, a refund is approved, or the guest is a no-show or checks out early; otherwise it ends once the night has passed and is kept as stay history. In the database a night on sale has no row: holding, booking or blocking creates one and releasing the night deletes it.

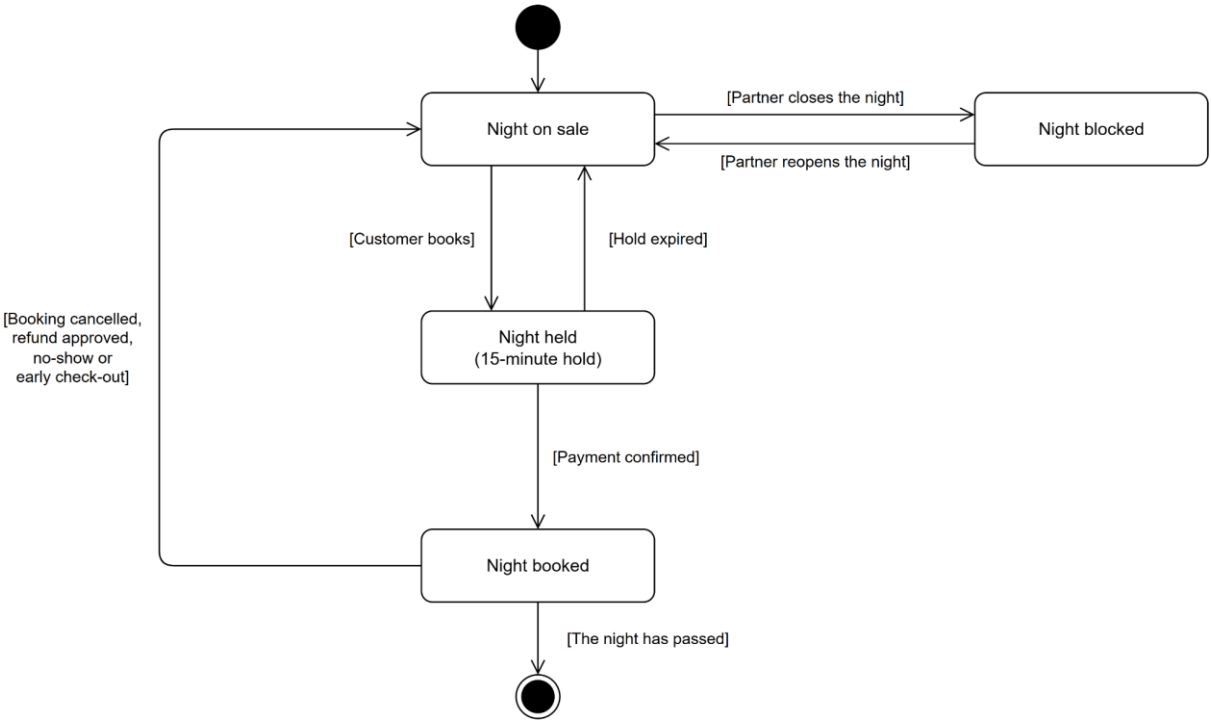


Figure 22. State machine diagram of room night

4.2.7 Booking

A new booking holds its rooms for 15 minutes; if it is not paid in time, the hold expires. When payment is received, the invoice is marked paid and the notification and email are sent in parallel, then the booking is confirmed. A confirmed booking ends in one of three ways: the guest checks in and later checks out, the guest does not arrive and is marked as a no-show, or the guest cancels. A cancellation with no refund due cancels the booking at once. Otherwise the refund waits for the owner: the booking is cancelled when the owner approves or the request is auto-approved at the check-in time, and stays confirmed when the owner rejects it. There is no automatic check-out.

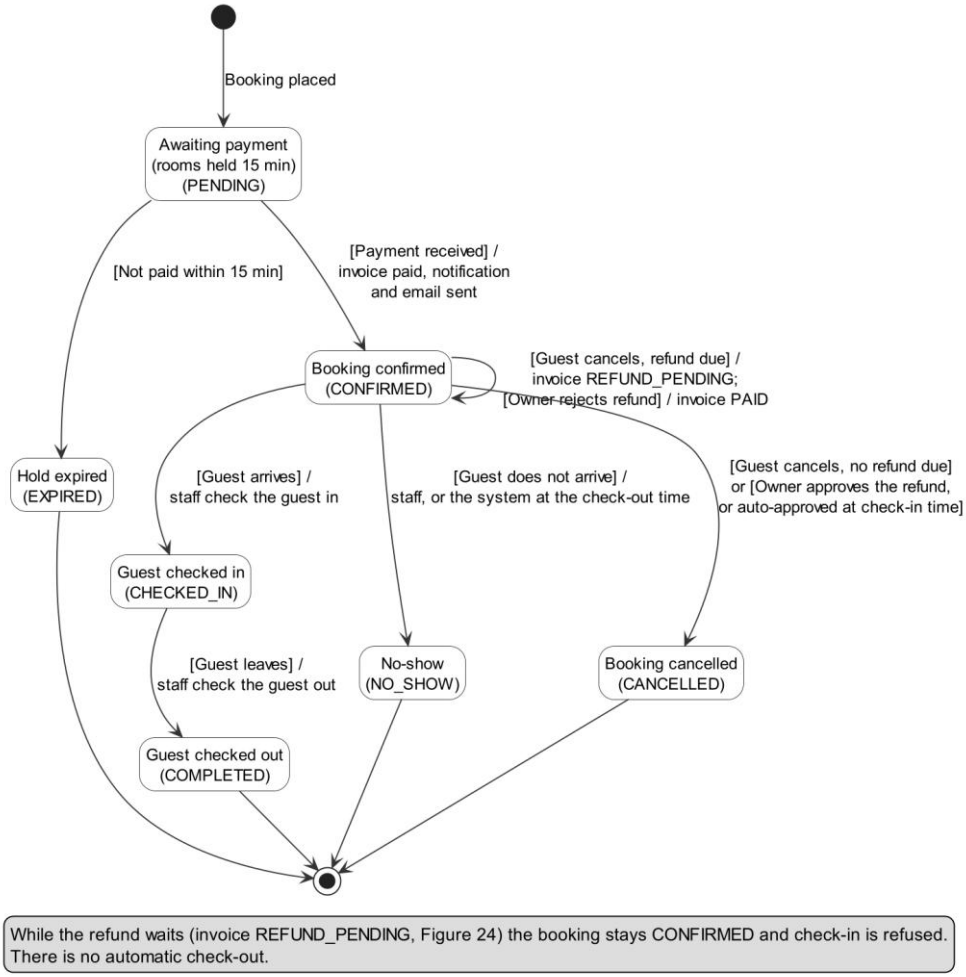


Figure 23. State machine diagram of booking

4.2.8 Payment status of the invoice

The invoice waits for payment from the moment the booking is created. It becomes paid when the SePay webhook confirms the transfer, or fails when the hold expires. If the customer cancels with a refund, the invoice moves to refund pending; a request the partner has not decided by the check-in time is approved automatically. Approval ends in a full refund or a partial refund after the cancellation fee, while rejection returns the invoice to paid.

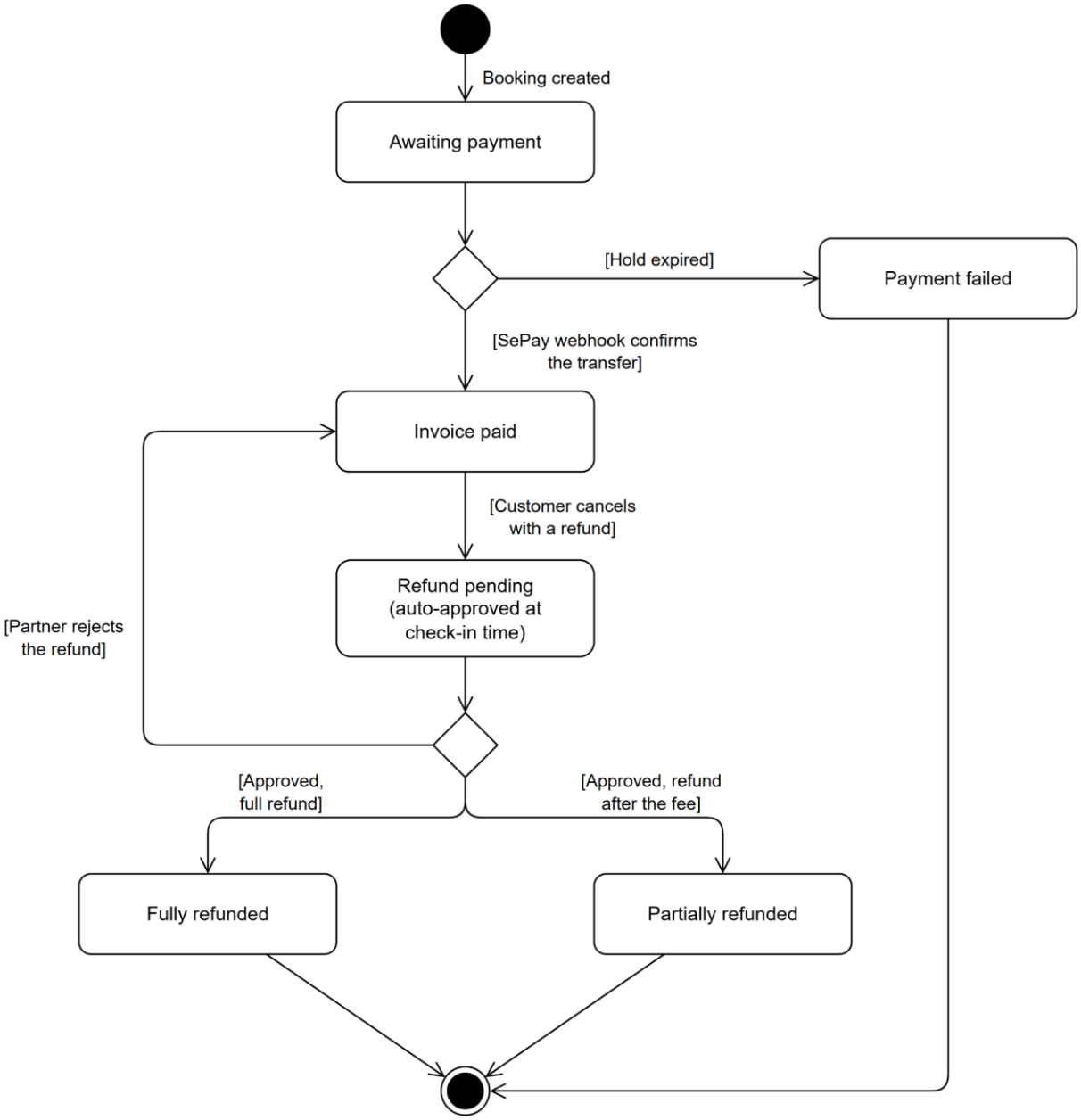


Figure 24. State machine diagram of payment status of the invoice

4.2.9 Marketing content

This is the planned life cycle of an AI-assisted post in WF-05, so every transition is dashed. Hotel staff draft the post with AI and submit it for partner approval. A rejected post goes back to staff for editing. An approved post is either published at once or scheduled and published when the set time is reached.

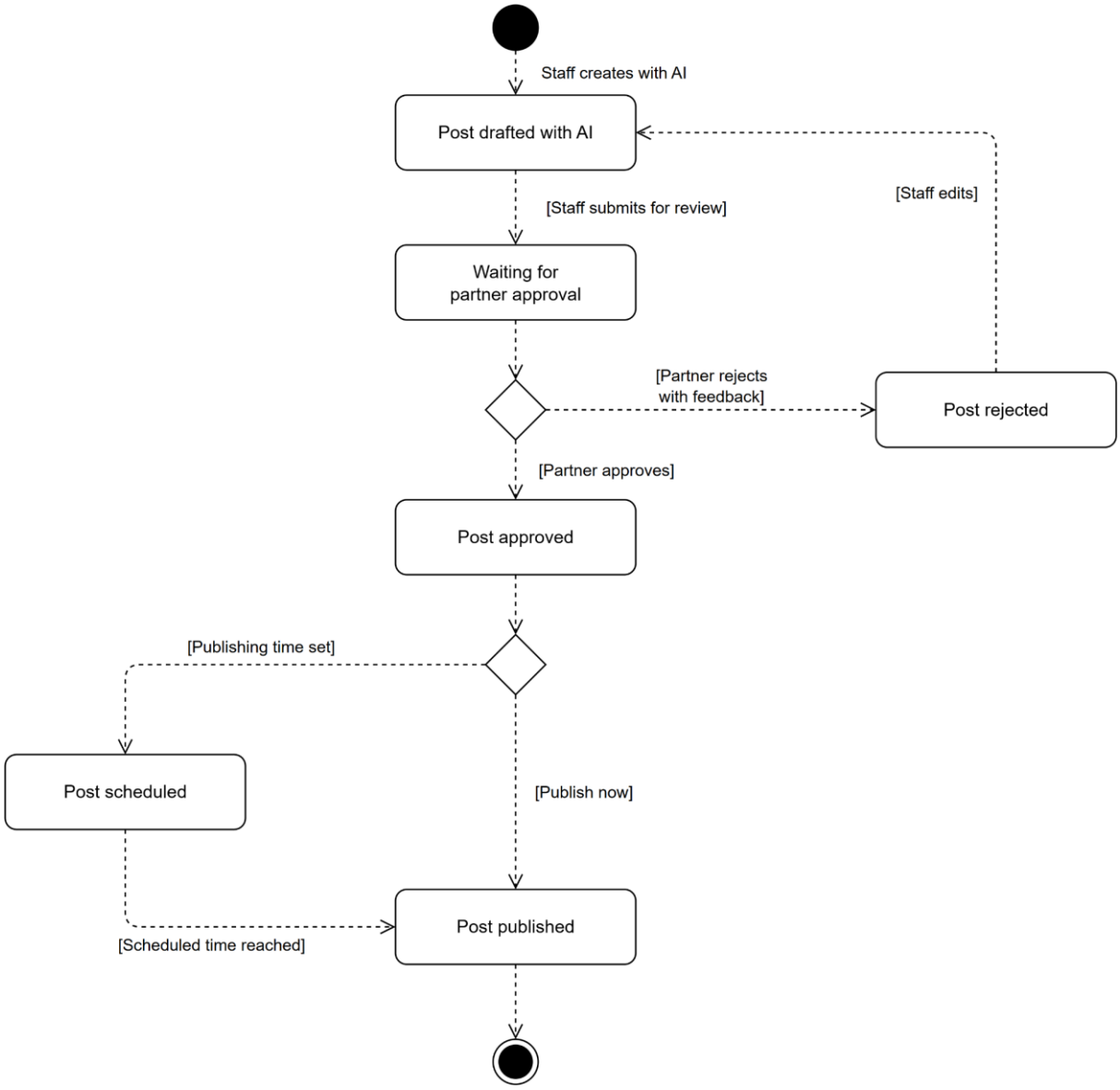


Figure 25. State machine diagram of marketing content